İÇ TEKNİK RAPOR

25 Eylül 2026

Gerçek Zamanlı Fizyolojik Stres Tespitinde Açıklanabilirliğin Hesaplama Maliyeti

Uç, Sis ve Bulut Yerleşiminde Uçtan Uca Ölçüm ve Güven Eşikli Seçici Açıklama

Bu rapor ne yapar. Depodaki *bütün* kodu, *bütün* deneyi ve *bütün* sonucu tek belgede toplar. Her sayının nereden geldiği, hangi koşulda ölçüldüğü ve ne anlama geldiği yazılıdır. Kullanılan her teknik terim Bölüm 2'deki sözlükte tanımlanmıştır — rapor, alanı bilmeyen birinin baştan sona okuyup anlayabileceği biçimde yazıldı.

Kaynak. Bütün sayılar `results/` altındaki JSON ve CSV dosyalarından okunmuştur. Elle taşınan, yuvarlanan veya tahmin edilen sayı yoktur; türetilmiş (ölçülmemiş) değerler açıkça *projeksiyon* diye işaretlenmiştir.

İçindekiler

1 Bir Bakışta **4**

1.1 Çalışmanın tek cümlelik özeti . . . 4

1.2 Ölçülen dört büyük sayı . . . 4

1.3 Bu raporu nasıl okumalı . . . 5

2 Sözlük: Her Terimin Anlamı **6**

2.1 Sinyal ve veri terimleri . . . 6

2.2 Normalizasyon terimleri . . . 7

2.3 Makine öğrenmesi terimleri . . . 7

2.4 Açıklanabilirlik (XAI) terimleri . . . 9

2.5 Dağıtık sistem ve ölçüm terimleri . . . 11

3 Sistem Mimarisi ve Kod Haritası **15**

3.1 Veri akışı . . . 15

3.2 Dosya dosya kod haritası . . . 15

3.3 Ölçümü mümkün kılan üç tasarım kararı . . . 17

4 Model ve Model Seçimi **18**

4.1 Dağıtılan model: TemporalCNN . . . 18

4.2 Neden en yüksek doğruluklu model değil? . . . 18

5 Deneyler I: Sistem Profili ve Ölçüm Altyapısı **20**

5.1 01_streaming — tek süreçli replay, aşama profili . . . 20

5.2 02_benchmark — tekrarlı ve dönüşümlü ölçüm . . . 21

5.2.1 İş parçacığı taraması . . . 21

5.2.2 İki ardışık düzen yolu: `prepared` ve `native` . . . 21

5.2.3 Açıklayıcıların istemci makinesindeki CPU maliyeti . . . 22

5.3 03_loopback — istemci ve sunucu ayrı süreçte, aynı makinede . . . 22

5.4 05_yerlesim — ortak yerleşim (uç) ve sis karşılaştırması . . . 23

5.5 12_profil — aşama profilinin dört kaynaktan derlemesi . . . 23

6 Deneyler II: Açıklamanın Maliyeti **25**

6.1 06_xai_tarama — açıklayıcı hiperparametre taraması . . . 25
6.2 04_fog — makalenin ana tablosu . . . 26
6.3 07_edge_xai — aynı beş durum, ortak yerleşimde . . . 28
6.4 13_bulut — bulut yerleşimi . . . 28
6.5 11_pareto — kalite ile gecikmenin dengesi . . . 29

7 Deneyler III: Seçici Açıklama 31
7.1 08_kalibrasyon — seçici açıklamanın ön koşulu . . . 31
7.1.1 Sıcaklık ölçekleme çalışmadı . . . 31
7.1.2 Aday eşikler (düzeltmesiz) . . . 32
7.2 09_secici_xai — ana sonuç . . . 32
7.2.1 Tam kayıt koşusu . . . 32
7.2.2 Projeksiyon ölçümle doğrulandı . . . 33
7.2.3 Eşiğin hiç tetiklenmediği üç koşu . . . 33
7.3 10_yuk — çok istemcili yük . . . 34

8 Şekiller ve Nasıl Okunmalı 36

9 Donanım Fizibilitesi: Model Bir Mikrodenetleyicide Çalışır mı? 40

10 Bitirme Projesinden Gelen Sonuçlar 42
10.1 `Transformer.ipynb` — TranSenseFusers . . . 42
10.2 `XAİ_metric_transensefusıon.ipynb` — offline XAI kalitesi . . . 42

11 Geçersiz Sayılan Ölçümler ve Nedenleri 44
11.1 Saat kayması düzeltilmeden önceki sis koşusu . . . 44
11.2 Sızıntılı normalizasyonla ölçülen XAI taramaları . . . 44
11.3 `prepared` yol dönemi streaming sonuçları . . . 45
11.4 Duman testleri . . . 45

12 Tespit Edilen Tutarsızlıklar 46

13 Sentez: Bütün Sonuçlar Bir Arada 47
13.1 Sistemin tam maliyet resmi . . . 47
13.2 Neden tek kullanıcıda bile önemli . . . 47
13.3 Referans makale ile karşılaştırma . . . 48

14 Sınırlılıklar 49

15 Ekler 50
15.1 Her sayı hangi dosyadan . . . 50
15.2 Deneyleri yeniden koşma komutları . . . 51

15.3 Docker ile sunucuyu dağıtma 51
15.4 Bu raporu yeniden derleme 52

Bölüm 1

Bir Bakışta

1.1 Çalışmanın tek cümlelik özeti

Ana bulgu

Gerçek zamanlı bir stres tespit sisteminde **darboğaz modelin tahmini değil, o tahmini açıklamaktır**. Sis düğümünde bir 10 saniyelik pencerenin tahmini 3,39 ms sürüyor; aynı pencerenin bir SHAP veya LIME açıklaması 395–1422 ms sürüyor, yani **toplam hesabı 118–421 kat** büyütüyor. Bunun sonucu olarak sistem tasarımındaki kritik karar *"hesabı nerede yapayım"* (uç/sis/bulut) olmaktan çıkıp *"ne zaman açıklayayım"* olmaya kayıyor.

Önerilen çözüm ve ölçülen kazancı

Güven eşikli seçici açıklama: model kendinden eminse (güven $\geq 0{,}80$) yalnızca tahmin döndür, emin değilse tahmin + açıklama döndür. 1030 pencerelik gerçek bir koşuda ölçüldü:

- Ortalama yanıt süresi 450,5 ms $\rightarrow$ 80,1 ms (**5,6 kat ucuz**)
- Pencerelerin yalnızca %14,1'i açıklanıyor
- Buna rağmen yanlış sınıflanan pencerelerin **%89,7'si** hâlâ açıklama alıyor
- Tek işçili kapasite 12 $\rightarrow$ 90 eşzamanlı istemci (**7,5 kat**)

1.2 Ölçülen dört büyük sayı

Bulgu	Sayı	Kaynak
Açıklamanın toplam hesabı büyütme oranı	118–421×	`04_fog/`
Yerleşim farkı, açıklama *kapalıyken* (bulut / uç)	8,7×	`13_bulut/`
Yerleşim farkı, açıklama *açıkken* (bulut / uç, SHAP)	1,13×	`13_bulut/`
Seçici açıklamanın ortalama yanıt süresinde kazancı	5,6×	`09_secici_xai/`

Tablo 1.1: Raporun taşıdığı dört sayı. Üçüncü satır ikinciyle birlikte okunmalı: açıklama açıldığı anda 50 ms'lik ağ farkı 1200 ms'lik açıklamanın yanında kayboluyor.

1.3 Bu raporu nasıl okumalı

- **Bölüm 2 (Sözlük)** kullanılan her terimi tanımlar. Metinde bir terim ilk geçtiğinde kalın yazılmıştır; anlamını bilmiyorsan sözlüğe bak.
- **Bölüm 3 (Sistem)** kodun mimarisini ve hangi dosyanın ne yaptığını anlatır.
- **Bölüm 4 (Model)** sınıflandırıcıyı ve neden bu modelin seçildiğini anlatır.
- **Bölüm 5–7** deneyleri tek tek ele alır. Her deney aynı kalıpla yazıldı: *amaç* → *nasıl ölçüldü* → *tablo* → *ne anlama geliyor* → *uyarılar*.
- **Bölüm 8** şekilleri tek tek okur.
- **Bölüm 9–11** geçersiz sayılan ölçümleri, tespit edilen tutarsızlıkları ve sınırlılıkları listeler. Bunlar makale yazarken ve hakem sorusu geldiğinde en çok işine yarayacak bölümlerdir.

Bölüm 2

Sözlük: Her Terimin Anlamı

Bu bölüm referans amaçlıdır; baştan sona okumak zorunda değilsin. Raporun geri kalanında geçen her teknik terim burada tanımlıdır.

2.1 Sinyal ve veri terimleri

Veri ve pencereleme

WESAD
Wearable Stress and Affect Detection. 15 katılımcıdan (S2–S17, S12 yok) göğüs (RespiBAN) ve bilek (Empatica E4) cihazlarıyla toplanmış fizyolojik sinyal veri kümesi. Bu çalışmada **ikili** problem kullanılıyor: *stres* (etiket 2) ve *stres değil* (etiket 1, 3, 4 — taban çizgi, eğlence, meditasyon).

Kanal
Tek bir sensörün tek bir ölçüm ekseni. Bu çalışmada **14 kanal** var: göğüsten 8 (ACC x/y/z, ECG, EMG, EDA, Temp, Resp), bilekten 6 (ACC x/y/z, BVP, EDA, TEMP).

ACC / ECG / EMG / EDA / BVP / Resp / Temp
Sırasıyla ivmeölçer (hareket), elektrokardiyogram (kalp elektriksel etkinliği), elektromiyogram (kas etkinliği), elektrodermal aktivite (deri iletkenliği — ter bezleri, stresin klasik göstergesi), kan hacmi nabzı (optik nabız), solunum, sıcaklık.

Örnekleme hızı (sampling rate)
Bir sensörün saniyede kaç ölçüm ürettiği. Göğüs kanallarının hepsi 700 Hz, bilek ACC 32 Hz, bilek BVP 64 Hz, bilek EDA ve TEMP 4 Hz. Sistem hepsini ortak **64 Hz**'e indirir.

Kanal sırası
Modelin gördüğü 14 kanalın sırası. *Kritiktir:* sıra bozulursa model karışık kanal alır ve doğruluk çöker (1,00 yerine 0,52 gözlendi). Doğru sıra önce göğüs (8), sonra bilek (6).

Pencere (window)
Modele tek seferde verilen zaman dilimi. Burada **10 saniye** = 64 Hz $\times$ 10 s = **640 örnek**. Modelin girdisi bu yüzden 14×640'lık bir matristir.

Kayma / adım (hop, stride)
İki ardışık pencerenin başlangıcı arasındaki süre. Burada **5 saniye** = 320 örnek, yani **%50 örtüşme** (overlap). Sonuç: sistem her 5 saniyede bir yeni tahmin üretir.

Tick
İstemcinin sunucuya bir seferde gönderdiği veri bloğunun süresi. Burada **500 ms**. 600 tick = 5

dakika, 10 466 tick = 87 dakika.

Yeniden örnekleme (resampling)
Farklı hızlardaki sinyalleri ortak hıza (64 Hz) indirme işlemi. Doğrusal ara değerle (`np.interp`) yapılıyor. Bu, ölçülen bir maliyettir: toplam hesabın **%28,6**'sı.

Bant genişliği (bandwidth)
Ağdan saniyede geçen bit sayısı. Burada istemcinin gönderdiği + aldığı bayt / geçen süre, kbps cinsinden.

2.2 Normalizasyon terimleri

Normalizasyon

z-score normalizasyonu
Her kanalı $(x - \mu)/\sigma$ ile ölçekleme. Model eğitimde bunu gördüğü için dağıtımda da aynı ölçekte veri almalı.

Denek bazlı (subject-wise) normalizasyon
Her deneğin *tüm* kaydının ortalaması ve standart sapmasıyla normalize etmek. Offline eğitimde kullanıldı. Gerçek zamanlı sistemde **kullanılamaz**, çünkü henüz gelmemiş verinin istatistiğini bilmek gerekir.

Sızıntı (leakage)
Bir modelin, dağıtımda erişemeyeceği bilgiyi kullanması. "Geleceğe sızmak" burada tam olarak denek bazlı normalizasyonu gerçek zamanlı ölçüme sokmak demektir; sonuçları olduğundan iyi gösterir.

Nedensel (causal) normalizasyon — `running`
Yalnızca *o ana kadar görülmüş* veriden hesaplanan çevrimiçi ortalama/standart sapma. Denek bazlı z-score'un sızıntısız karşılığıdır. Varsayılan moddur.

Welford algoritması
Ortalama ve varyansı tek geçişte, sayısal olarak kararlı biçimde güncelleyen yöntem. `RunningNormalizer` bunu kullanır; her tick'te güncelleme maliyeti dürüstlük için *preprocessing* süresine eklenir.

Sabit (fixed) normalizasyon
Yalnızca *eğitim* deneklerinden bir kez hesaplanmış tek bir ortalama/standart sapma. En katı dağıtım senaryosu: yeni kullanıcı hakkında hiçbir şey bilinmiyor.

2.3 Makine öğrenmesi terimleri

Model ve başarım

1B Evrişimli Sinir Ağı (1D-CNN)
Zaman serisi üzerinde kayan filtrelerle örüntü arayan ağ. Burada `TemporalCNN`: iki evrişim bloğu (Conv1d $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Dropout), sonra küresel ortalama havuzlama ve iki tam bağlı katman.

Filtre / kernel / depth
Filtre sayısı bir katmanın kaç farklı örüntü aradığı ([32, 128]); kernel boyutu filtrenin kaç örneklik pencereye baktığı (9, yani ≈140 ms); depth blok sayısı (2).

BatchNorm
Katman çıktısını normalize eden katman. `eval()` modunda koşu istatistiklerini kullanır, yani parti boyutu sonucu değiştirmez — bu, XAI'nin yüzlerce perturbasyonu partiler hâlinde gönderebilmesinin ön koşuludur.

Dropout
Eğitimde rastgele nöron kapatarak ezberlemeyi azaltan yöntem. Çıkarımda kapalıdır.

Focal loss
Kolay örneklerin katkısını bastırıp zor örneklere ağırlık veren kayıp fonksiyonu; sınıf dengesizliği için. Burada $\gamma = 2$, $\alpha = 0{,}5$.

Parametre sayısı
Modelin öğrenilen ağırlık sayısı. `TemporalCNN`: **49 890**. Karşılaştırma: TranSenseFusers 1 859 026.

MAC (çarpma-toplama)
Bir çıkarımın gerektirdiği temel aritmetik işlem sayısı; hesap yükünün donanımdan bağımsız ölçüsü. Burada pencere başına **14,50 milyon**.

LOSO (leave-one-subject-out)
Her turda bir deneği tamamen dışarıda bırakıp kalanlarla eğitme, sonra o denekte test etme; 15 denek için 15 tur. Denek-bağımsız başarımı ölçen en dürüst protokol. *Tek denek dışarıda bırakmak LOSO değildir* — rapor bu ayrımı birkaç yerde vurgular.

Tutulmuş denek (held-out subject)
Modelin hiç görmediği denek. Burada S11 ve S14. Gerçek zamanlı ölçümlerin hepsi bu iki denekte yapılır.

Doğruluk (accuracy)
Doğru tahmin / toplam tahmin.

Kesinlik (precision)
Stres dedin, gerçekten stres miydi? TP/(TP+FP).

Duyarlılık (recall)
Gerçek streslerin kaçını yakaladın? TP/(TP+FN).

F1
Kesinlik ve duyarlılığın harmonik ortalaması. *Weighted F1* sınıfların büyüklüğüne göre ağırlıklandırılmış hali.

Karışıklık matrisi
TN (doğru negatif), FP (yanlış alarm), FN (kaçırılan stres), TP (yakalanan stres).

Softmax ve güven (confidence)
Modelin çıktısını olasılığa çeviren fonksiyon; *güven* = en yüksek sınıf olasılığı. İkili problemde 0,5 ile 1,0 arasındadır. Seçici açıklamanın tetikleyicisi budur.

2.4 Açıklanabilirlik (XAI) terimleri

Açıklayıcılar

Atıf (attribution)
"Girdinin hangi kısmı bu kararı ne kadar etkiledi?" sorusuna sayısal cevap. Burada girdiyle aynı şekilde, yani 14×640'lık bir matris.

SHAP
SHapley Additive exPlanations. Oyun kuramındaki Shapley değerinden türetilmiş atıf yöntemi: her girdi öğesinin katkısı, o öğe olmadığında sonucun ne kadar değiştiğine bakılarak hesaplanır.

GradientExplainer
SHAP'ın **gradyan tabanlı** uygulaması. Modelin içine erişir (türev alır), bu yüzden hızlıdır. *Bu çalışmada kullanılan budur.*

KernelSHAP
SHAP'ın model-agnostik, perturbasyon tabanlı uygulaması. Literatürdeki "SHAP çok yavaştır" iddiası *buna* aittir; bu raporun sayıları KernelSHAP'a ait değildir. Karıştırılmamalıdır.

Arka plan kümesi (background set)
SHAP'ın "bu değer olmasaydı ne olurdu" sorusunu sorarken referans aldığı pencere kümesi. Burada eğitim deneklerinden (S2–S5) k-means ile seçilmiş **100** pencere; test denekleriyle kesişmiyor. Dosyaya kaydedilmiştir, böylece sunucu her açılışta yeniden kümeleme yapmaz.

`nsamples`
GradientExplainer'ın arka plan ile girdi arasında kaç nokta örneklediği. Maliyeti doğrudan belirler: 200 (tam) ve 50 (ucuz).

LIME
Local Interpretable Model-agnostic Explanations. Girdinin parçalarını rastgele kapatıp modelin cevabındaki değişimi gözler, sonra bu gözlemlere basit bir doğrusal model uydurur; o doğrusal modelin katsayıları atıf olur.

Bölge tabanlı LIME (region-based LIME)
Hazır `lime` paketi kullanılmadı. Girdi $14 \times 640 = 8960$ değer; her örneği ayrı değişken saymak hem çok pahalı hem yorumlanamaz. Görüntüdeki "süper piksel"in karşılığı burada **zaman dilimi** $\times$ **kanal** bölgeleridir: **20 dilim** $\times$ **14 kanal = 280 bölge**.

Perturbasyon (perturbation) / örnek sayısı
LIME'ın ürettiği "bazı bölgeleri kapatılmış" pencere sayısı. Maliyet bununla doğrusal büyür: 500 (tam) ve 250 (ucuz).

Maskeleme / taban çizgi (baseline)
Bir bölge "kapatıldığında" yerine ne konur. Burada **o kanalın kendi ortalaması**. Sıfır konulmazdı: normalize edilmiş sinyalde sıfır "bilgi yok" demek değil, "kaydın ortalaması" demektir ve fizyolojik olarak anlamsızdır (0 °C sıcaklık gibi).

Kernel genişliği (kernel width)
LIME'da, orijinale yakın perturbasyonlara ne kadar ağırlık verileceğini belirleyen parametre. Burada 0,5.

Ridge regresyon
LIME'ın uydurduğu, küçük katsayıları cezalandıran doğrusal model.

Integrated Gradients (IG)
Üçüncü bir atıf yöntemi; bitirme projesinde kullanıldı, bu depoda *yok*.

Açıklama kalitesi ölçütleri

Deletion eğrisi ve AUC
En önemli bölgelerden başlayarak kapat, her adımda modelin olasılığını ölç. Açıklama sadıksa olasılık hızla düşer, yani **düşük AUC iyi**.

Insertion eğrisi ve AUC
Taban çizgiden başlayıp en önemli bölgeleri ekle. Açıklama sadıksa olasılık hızla yükselir, yani **yüksek AUC iyi**.

Faithfulness (sadakat)
$=$ insertion AUC $-$ deletion AUC. Açıklama gerçekten modelin kullandığı yeri mi gösteriyor? **Yüksek iyi.**

Monotonicity (tekdüzelik)
Insertion eğrisinde azalmayan adımların oranı. Önem sırası doğruysa her ekleme olasılığı artırmalı. **Bu depoda yüksek iyi —** bitirme projesindeki tanım farklıdır (orada ihlal sayılıyor ve düşük iyi), ikisi karşılaştırılamaz.

Sensitivity (girdi duyarlılığı)
Girdiye küçük gürültü eklenince atıfın ne kadar değiştiği (normalize L2). **Düşük = kararlı açıklama.**

Self-instability (kendi kararsızlığı)
Aynı pencere iki kez açıklandığında çıkan fark. Yöntemin kendi rastgeleliğini ölçer. Seed sabitlendikten sonra ikisinde de ≈ 0; sabitlenmeden önce SHAP'ın sensitivity'sinin neredeyse yarısı bu gürültüydü (0,139 $\rightarrow$ 0,075).

Complexity / Gini / sparseness
Atıfın ne kadar az sayıda öğede yoğunlaştığı. Bitirme projesinde ölçüldü; bu depoda yok.

SEM (standart hata)
Ortalamanın belirsizliği: $\sigma/\sqrt{n}$. İki ölçüm arasındaki fark SEM mertebesindeyse "fark yok" denmelidir. Bu rapor faithfulness'ta tam olarak bu durumdadır.

2.5 Dağıtık sistem ve ölçüm terimleri

Yerleşim

Uç düğüm / sensör
Giyilebilir cihazın kendisi (ör. STM32 sınıfı mikrodenetleyici).

Edge — ortak yerleşim (co-located)
Bu raporda "edge", sunucunun istemciyle aynı makinede çalıştığı durumdur, giyilebilir cihaz değil. Ağ yok, TCP loopback üzerinden konuşurlar.

Fog (sis)
Aynı yerel ağdaki ikinci bir fiziksel makine. Burada 192.168.1.14 (Intel 12 çekirdek, 16 GB), Wi-Fi üzerinden.

Cloud (bulut)
İnternet üzerindeki uzak sunucu. Burada bir DigitalOcean VPS (167.71.43.64, 1 çekirdek, 2 GB), Docker konteynerinde.

Worker (işçi)
Modeli çalıştıran görev. Bu sistemde **tek** worker var; bu, referans makaleyle aynı varsayım ve kuyruk gecikmesinin ölçülebilmesinin ön koşuludur.

Broker
İstekleri birden fazla worker'a dağıtan yönlendirici. Bu sistemde *yok*; tek worker varken dağıtacak bir şey olmadığı için sadece fazladan sıçrama eklerdi.

Açık istek kuyruğu
Okuyucu görev paketi alır, varış zamanını damgalar ve ortak kuyruğa koyar; worker kuyruğu sırayla boşaltır. İlk sürümde paket okunur okunmaz işleniyordu ve bekleme süresi gecikmenin içinde gizleniyordu.

Loopback
Aynı makinede 127.0.0.1 üzerinden TCP. Gerçek ağ yok ama protokol maliyeti var.

Yedi ölçülen metrik

1. Latency (ağ gecikmesi)
Paketin istemciden sunucuya *tek yönde* ulaşma süresi. İki farklı saat kullandığı için saat farkı düzeltmesi gerektirir; ölçümü en kırılgan metrik budur.

2. Queuing delay (kuyruk gecikmesi)
Worker meşgulken paketin kuyrukta beklediği süre. Sunucu saatiyle ölçülür, saat farkından etkilenmez.

3. Execution time (yürütme süresi)
Sunucuda ardışık düzenin çalışma süresi: yeniden örnekleme + normalizasyon + tensör hazırlığı + çıkarım + son işleme + *(varsa)* açıklama. Sunucu saati.

4. Response time (yanıt süresi)
İstemcinin gördüğü toplam süre: gönderdiği andan cevabı aldığı ana kadar. İstemci saati. Yaklaşık olarak $2\times$latency + queuing + execution.

5. Jitter (titreşim)
Ardışık yanıt sürelerinin ortalama mutlak farkı. Sistemin ne kadar "düzensiz" olduğunu gösterir.

6. Bandwidth
Yukarıda tanımlandı.

7. Resource usage
Sunucu sürecinin CPU yüzdesi ve RSS belleği.

Ölçüm metodolojisi

p50 / medyan
Değerlerin yarısı bunun altında. Ortalamadan daha dayanıklıdır (tek bir uç değer bozmaz).

p95 / p99
Değerlerin %95'i / %99'u bunun altında. "En kötü durum" davranışı. Gerçek zamanlı sistemde ortalamadan daha önemlidir.

Warm-up (ısınma)
İlk pencereler önbellek/CUDA ısınması yüzünden yanıltıcıdır. İlk **15** pencere hiçbir istatistiğe girmez.

Koşu içi dağılım
Bir koşudaki yüzlerce pencerenin p50/p95/p99'u. Sistemin en kötü durum davranışını gösterir.

Koşu arası yayılım
Aynı deney tekrarlandığında medyanın oynaması. Ölçümün güven aralığıdır. İkisi karıştırılmamalıdır.

Dönüşümlü tekrar (alternating repeats)
A, B, A, B sırasıyla koşmak. Ardışık koşulsaydı işlemci ısınması son ölçülen yapılandırmayı sistematik olarak yavaş gösterirdi.

CV (değişim katsayısı)
Standart sapma / ortalama, yüzde olarak. *range_pct* ise (max–min)/medyan.

Saat senkronizasyonu
İki makinenin yüksek çözünürlüklü sayaçları bağımsızdır. Bağlantı başında NTP mantığıyla fark ölçülür (12 tur, en düşük gidiş-dönüş süreli tur seçilir).

`perf_counter` ve neden `time_ns` değil
Windows'ta `time.time_ns()` çözünürlüğü 15,6 ms ölçüldü; loopback gecikmesi 0,1 ms mertebesinde olduğu için latency medyanı tam sıfır okuyordu. `perf_counter` 100 ns verir ama referans noktası sürece özeldir — bu yüzden saat farkı ölçümü zorunludur.

Kayma (drift) ve ppm
İki kristal aynı hızda tiklemez; fark zamanla kayar. 87 dakikalık bir koşuda kayma latency medyanını negatife düşürmüştü. Şimdi saat farkı her **60 saniyede** yeniden ölçülüyor ve her paketin latency'si iki ölçüm arasında doğrusal ara değerle düzeltiliyor. *ppm* = milyonda bir oranında kayma hızı.

Belirsizlik (uncertainty)
Tek yönlü gecikme tahmininin hata payı; gidiş-dönüş süresinin yarısı. Loopback'te $\pm$0,13 ms, Wi-Fi'da $\pm$1–1,7 ms, internette $\pm$25–51 ms.

RSS (resident set size)
Sürecin fiziksel bellekte tuttuğu miktar, MB.

Kapasite
Hop bütçesi (5000 ms) $\div$ execution time. Bir worker'ın kaç eşzamanlı istemciyi gerçek zamanda taşıyabileceğinin üst sınırı.

Kalibrasyon terimleri

Güven kalibrasyonu
Modelin "%90 eminim" dediği pencerelerin gerçekten %90'ında haklı olması. Seçici açıklamanın *ön koşuludur*: güven gerçek doğruluk olasılığını temsil etmiyorsa eşik anlamsız olur.

Reliability diyagramı
Yatay eksen ortalama güven, dikey eksen gözlenen doğruluk. Mükemmel kalibre model köşegen üzerindedir.

ECE (expected calibration error)
Kutulardaki |güven − doğruluk| farkının, kutu büyüklüğüyle ağırlıklı ortalaması. **Düşük iyi.**

MCE (maximum calibration error)
Aynı farkın en büyüğü.

Brier skoru
Olasılık tahmininin ortalama kare hatası. Düşük iyi.

NLL (negative log-likelihood)
Olasılık tahmininin log-kayıp değeri. Düşük iyi.

Fazla özgüven (overconfidence)
Ortalama güven $-$ doğruluk. Pozitifse model olduğundan emin.

Sıcaklık ölçekleme (temperature scaling)
Softmax'tan önce logitleri tek bir T sayısına bölerek kalibrasyonu düzeltme. $T > 1$ güveni düşürür. Tek parametrelidir ve doğruluğu *değiştirmez*.

Açıklama oranı (explanation rate)
Açıklanan pencere / toplam pencere.

Hata yakalama (error recall)
Yanlış sınıflanan pencerelerin kaçında açıklama tetiklendi. Seçici açıklamanın *faydasını* ölçer.

Tetikleme isabeti (precision of trigger)
Açıklanan pencerelerin kaçı gerçekten hatalıydı.

Bölüm 3

Sistem Mimarisi ve Kod Haritası

3.1 Veri akışı

```
WESAD .pkl (istemcide)
    ↓ ham hızlarda (göğüs 700 Hz, bilek 64/32/4 Hz)
İSTEMCİ: 500 ms'lik tick blokları → TCP paketi (t_send damgası)
    ↓ ağ (loopback / Wi-Fi / internet)
SUNUCU okuyucu görevi: paketi al, t_recv damgala, KUYRUĞA koy
    ↓ (kuyrukta bekleme = queuing delay)
SUNUCU worker (tek):
    1. MultiRateBuffer.push → pencere hazır mı?
    2. resampling: her akışı 64 Hz'e indir → (640, 14)
    3. preprocessing: Welford güncelle + nedensel z-score
    4. tensor_prep: (640,14) → torch (1,14,640)
    5. inference: TemporalCNN → logits
    6. postprocessing: softmax → pred, confidence
    7. SEÇİCİ XAI: confidence < eşik ise açıklayıcıyı çalıştır
    ↓ (1--7 toplamı = execution time)
Cevap paketi (t_start, t_end, pred, confidence, stages, explained)
    ↓
İSTEMCİ: t_reply − t_send = response time
```

Kritik tasarım kararı

Açıklama, tahminle **aynı worker'da** ve **execution time'ın içinde** üretilir. Ayrı bir ipliğe atılsaydı maliyeti görünmez olurdu; deneyin bütün amacı o maliyeti ölçmektir.

3.2 Dosya dosya kod haritası

Dosya	satır	Ne yapar
`config.py`	138	Tek merkezi ayar: yollar, pencere/hop, kanal adları, model mimarisi, eğitim parametreleri, iplik sayısı, ardışık düzen yolu, normalizasyon modu. İplik ve pipeline kararlarının *gerekçeleri* yorum olarak burada.
`models/temporal_cnn.py`	106	`TemporalCNN` mimarisi ve `load_checkpoint()`. Checkpoint kendi mimarisini taşıdığı için config'e bakılmaz.

Dosya	satır	Ne yapar
`data/loader.py`	197	WESAD `.pkl` → 14 kanal. İki mod: ortak hıza indirilmiş (`load_wesad_subject`) ve **ham hızlarda** (`load_wesad_subject_native`). `NATIVE_STREAMS` tablosu burada.
`data/normalization.py`	132	Sabit istatistik hesabı/kaydı ve `RunningNormalizer` (Welford ile nedensel z-score).
`data/windowing.py`	50	Offline eğitim için pencereleme + alt-örnekleme.
`streaming/buffer.py`	156	`RollingBuffer` (tek hız) ve `MultiRateBuffer` (çok hız, pencere başına yeniden örnekleme).
`streaming/replay_simulator.py`	262	Bir deneği tick tick "replay" eder, her pencereyi profiler'a sokar.
`profiling/latency_profiler.py`	288	Aşama bazlı süre ölçümü + CPU/RAM örnekleyici + mean/median/p95/p99 raporu.
`serving/protocol.py`	245	Tel formatı, çerçeveleme, PING/PONG saat senkronizasyonu, paketleme/açma.
`serving/server.py`	569	Açık istek kuyruğu + tek worker, model, **seçici XAI**, donanım künyesi, kaynak ölçümü.
`serving/client.py`	493	Sensör istemcisi: ham hızda gönderir, zaman damgalarını kaydeder, saat farkını periyodik yeniden ölçer.
`xai/base.py`	81	Ortak `Explainer` arayüzü, pencere yönü çevrimi ((640,14)↔(14,640)), partili tahmin.
`xai/lime_.py`	99	Bölge tabanlı LIME: bölge ızgarası (önbellekli), maske örnekleme, kanal ortalamasıyla maskeleme, ağırlıklı Ridge.
`xai/shap_.py`	108	SHAP GradientExplainer, kayıtlı arka plan kümesiyle, seed sabit.
`xai/quality.py`	169	faithfulness / monotonicity / sensitivity, partili hesapla.
`xai/artifacts.py`	422	`xai_config.json` ve arka plan `.npz` üretimi/okunması, SHA-256 doğrulaması.
`xai/benchmark.py`	217	Açıklayıcıların CPU'da tekrarlı ölçümü.
`tools/calibration.py`	418	Reliability kutuları, ECE/MCE/Brier/NLL, sıcaklık ölçekleme, eşik tablosu.
`tools/selective_sweep.py`	266	Karışım modeliyle eşik projeksiyonu.
`tools/pareto.py`	240	Kalite–gecikme Pareto noktaları ve grafiği.
`tools/profile_table.py`	169	Dört kaynaktan aşama profili derlemesi.
`tools/figures.py`	407	Altı şeklin tamamını JSON'lardan üretir.
`tools/mcu_feasibility.py`	161	MCU bellek/hız fizibilitesi (donanımsız hesap).
`training/train_final_model.py`	204	Tek seferlik: checkpoint + sabit norm istatistikleri üretir.
`run_streaming_emulation.py`	181	Tek süreçli replay emülasyonu (ana script).
`run_fog_experiment.py`	299	Sunucuyu başlatır, N istemci koşturur, raporu toplar.
`benchmark_runs.py`	216	Tekrarlı, dönüşümlü ölçüm sürücüsü (iplik ve ardışık düzen taramaları).
`compare_deployments.py`	336	Aynı deneyi birden fazla yerleşimde dönüşümlü koşar, yayılımlı tablo üretir.
`wesad_xai_ofat_sweep_colab.py`	—	Colab/GPU üzerinde XAI hiperparametre taraması.

Dosya	satır	Ne yapar

Tablo 3.1: Toplam 6629 satır Python. `Transformer.ipynb` ve `XAİ_metric_transensefusıon.ipynb` bitirme projesinden gelen ayrı defterlerdir (Bölüm 10).

3.3 Ölçümü mümkün kılan üç tasarım kararı

1. **Saat katmanı `perf_counter_ns`, `time_ns` değil.** Windows'ta `time_ns` çözünürlüğü 15,6 ms ölçüldü ve latency metriğini tamamen anlamsızlaştırıyordu.
2. **Sunucuda açık istek kuyruğu.** İlk sürümde `t_start` ile `t_recv` bitişik olduğu için kuyruk gecikmesi her zaman sıfıra yakın okuyordu. Kuyruk eklendikten sonra iki istemciyle p99 kuyruk gecikmesi 0,37 ms'den 5,50 ms'ye çıktı — metrik artık gerçekten bekleme ölçüyor.
3. **Açıklayıcı açılışta ısıtılıyor.** SHAP'ın *ilk* çağrısı 12,4 saniye ölçüldü (grafik kurulumu ve önbellek ısınması), sonrakiler ≈0,85 saniye. Isıtılmasaydı bu 12 saniye ilk açıklamada tek worker'ı bloklar ve bütün kuyruk arkasında birikirdi. Ölçülen ısınma süreleri koşu künyelerinde: 1208–1251 ms (sis düğümü).

Bölüm 4

Model ve Model Seçimi

4.1 Dağıtılan model: TemporalCNN

Mimari	depth 2, filtreler [32, 128], kernel 9
Parametre	**49 890**
Girdi	14 kanal $\times$ 640 örnek (10 s @ 64 Hz)
Çıktı	2 sınıf (stres / stres değil)
Blok yapısı	Conv1d $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool(2) $\rightarrow$ Dropout(0,2), iki kez
Baş	GlobalAvgPool $\rightarrow$ FC(128$\rightarrow$64) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout(0,5) $\rightarrow$ FC(64$\rightarrow$2)
Eğitim	focal loss (α=0,5, γ=2), Adam lr=1e-3, wd=1e-4, batch 64, 150 epoch, patience 20, seed 42
Eğitim denekleri (13)	S2 S16 S7 S10 S4 S3 S17 S6 S9 S13 S15 S5 S8
Tutulmuş denekler (2)	**S11, S14** — replay için doğru seçim bunlar
Checkpoint	`wesad_best_model_f32-128_k9.pt`, val weighted F1 **0,9386**
Hesap	14 497 984 MAC / pencere; tepe aktivasyon 81 920 eleman (`bn2`)

Tablo 4.1: Dağıtılan sınıflandırıcı. Bütün gerçek zamanlı ölçümler bu model *değiştirilmeden* yapıldı.

Hangi başarım sayısı kanonik?

Aynı model için üç sayı dolaşıyor ve makalede **biri seçilip her yerde o kullanılmalı**:

- `config.py`: val weighted F1 **0,9386** — yalnızca iki denek üzerinde, LOSO değil. *Makalede kullanma.*
- 30 Ağustos taraması: LOSO F1 **0,9307** (depth2, k9)
- "Makale için sonuçlar": Dengeli CNN LOSO Acc **0,920** / F1 **0,920**

Öneri: LOSO sayısı, çünkü TranSenseFusers da LOSO diye raporlanmış — karşılaştırma ancak o zaman adil olur.

4.2 Neden en yüksek doğruluklu model değil?

Bitirme projesinde en yüksek doğruluğu TranSenseFusers (Temporal CNN + 8 başlı çapraz dikkat) verdi. Bu depoda buna rağmen 49,9K parametrelik CNN kullanılıyor. Gerekçe ölçüldü:

Model	Girdi	Parametre	Çıkarım (p50)	Doğruluk	Protokol
TranSenseFusers	14×7000 @700 Hz	1 859 026	103,7 ms	0,9113	tek denek (S3)
TemporalCNN	14×640 @64 Hz	**49 890**	**3,63 ms**	0,92	15 katlı LOSO
Oran		**37,3×**	**28,6×**	—	—

Tablo 4.2: Model seçimi. İkisi de aynı CPU'da, tek iplikte ölçüldü (25 Eylül 2026). TranSenseFusers'ın p95 çıkarımı 118,1 ms.

İki dürüstlük notu

1. %91,13 LOSO değil. `Transformer.ipynb` hücre 10'daki `Config`: `TEST_SUBJECT = 'S3'`, `VAL_SUBJECT = 'S16'`, kalan 13 denek eğitim. Yani bu sayı *tek bir* test deneğinin sonucu, 15 katlı ortalama değil. Doğrulama F1'i 1,0000 (S16) — tek denekte olağan ama kırılgan. Makalede bu sayıya "LOSO" **denmemeli**.

2. Pencere hızı da farklı. Transformer 700 Hz ham sinyalde (7000 örnek/pencere), CNN 64 Hz'de (640 örnek). 28,6× çıkarım farkının bir kısmı buradan geliyor ve tabloda "sampling rate" sütunu ile gösterilmeli.

Ek ayrıntı (hücre 10 çıktısından): TranSenseFusers 27. epoch'ta erken durdu; S3 üzerinde stres duyarlılığı 0,6220, stres kesinliği 0,9518, weighted F1 0,9040, makro F1 0,8492 — yani **yüksek doğruluk büyük ölçüde stres-olmayan sınıftan geliyor** (459 stres-olmayan, 127 stres).

Bölüm 5

Deneyler I: Sistem Profili ve Ölçüm Altyapısı

Bu bölümdeki deneyler "sistem ne kadar hızlı" sorusunu cevaplar; XAI henüz devrede değildir. Klasör numaraları depodaki `results/` yapısına karşılık gelir.

5.1 01_streaming — tek süreçli replay, aşama profili

Amaç. Ağ ve istemci–sunucu ayrımı olmadan, saf hesaplama maliyetinin aşama bazlı dökümünü almak.

Kurulum. S11 + S14, `native` yol (yeniden örnekleme ölçümün *içinde*), `running` normalizasyon, tick 500 ms, tek iplik, CPU. **2123 ölçülen pencere**, ilk 30 ısınma penceresi hariç.

Aşama	ortalama	medyan	p95	p99
resampling	1,507	1,378	2,328	3,259
preprocessing	0,251	0,218	0,401	0,924
tensor_prep	0,159	0,141	0,249	0,513
inference	2,951	2,682	3,997	5,720
postprocessing	0,108	0,086	0,171	0,380
toplam	**5,034**	**4,689**	**6,658**	**9,170**

Tablo 5.1: Aşama bazlı hesaplama süresi (ms). Jitter (ardışık fark ortalaması) 1,155 ms. Süreç CPU'su ortalama %90,1 (tepe %127,8), RSS ortalama 470,2 MB (tepe 558,0 MB). Kaynak: `01_streaming/streaming_summary_native_fixed.json`

n (etiketli)	doğruluk	stres kesinlik	stres duyarlılık	stres F1	TN	FP	FN / TP
1198	0,8222	0,5681	0,8930	0,6944	743	184	29 / 242

Tablo 5.2: Aynı koşunun sınıflandırma başarımı. Gerçek stres oranı %22,6, tahmin edilen %35,6 — model *stres tarafına eğilimli*: 184 yanlış alarma karşılık yalnızca 29 kaçırma.

Ne anlama geliyor

Offline weighted F1 0,9386'dan streaming doğruluğu 0,8222'ye düşüş **modelin değil, normalizasyon kısıtının sonucudur**: offline sayı denek bazlı (sızıntılı) z-score ile, streaming sayı nedensel (sızıntısız) z-score ile üretildi. Bu, raporlanması gereken dürüst bir maliyet.

5.2 02_benchmark — tekrarlı ve dönüşümlü ölçüm

Amaç. Tek koşu bu makinede güvenilir değil: işlemci frekans ölçeklemesi değerleri kaydırıyor. Bu dosyalar dönüşümlü tekrarla üretildi ve *koşu arası yayılımı* da taşıyor.

5.2.1 İş parçacığı taraması

iplik	toplam p50	yayılım (range)	CV	toplam p95
1	3,397 ms	**%2,9**	%1,2	4,980 ms
2	3,933 ms	%14,7	%6,0	5,774 ms
4	3,684 ms	%10,7	%4,5	5,508 ms

Tablo 5.3: S11, 587 pencere/koşu, 4 tur dönüşümlü. Kaynak: `02_benchmark/benchmark_thread_sweep.json`

Ne anlama geliyor

Medyanlar arasındaki fark (%15,8) koşu arası yayılımla *aynı mertebede*, yani hız farkı ayırt edilemiyor. Varsayılanın 1 olmasının gerekçesi hız değil **kararlılık**: tek iplikte ölçüm beş kat daha az oynuyor. Ayrıca giyilebilir/gömülü dağıtım senaryosuna en yakın olan bu.

5.2.2 İki ardışık düzen yolu: `prepared` ve `native`

Aşama	prepared p50	yayılım	native p50	yayılım
resampling	—	—	1,573	%3,3
preprocessing	0,580	%7,9	0,252	%3,5
tensor_prep	0,135	%5,7	0,149	%2,7
inference	2,969	%9,1	3,039	%8,9
postprocessing	0,085	%5,5	0,087	%3,0
toplam	**3,971**	%9,4	**5,502**	%8,1

Tablo 5.4: S11, 4 tur dönüşümlü, tek iplik, koşu içi medyanların medyanı (ms). Kaynak: `02_benchmark/benchmark_pipeline_cmp.json`

Ne anlama geliyor

`prepared` yolda sinyal replay başlamadan önce 64 Hz'e indirilmişti, yani yeniden örnekleme maliyeti ölçümün *dışında* kalıyordu. Gerçek bir sistemde sensör ham hızında gönderir ve indirgeme sunucuda yapılır, dolayısıyla `native` gerçekçi olan yoldur.
Yeniden örnekleme tek başına toplamın %28,6'sı (1,573/5,502), yani çıkarımdan sonraki en pahalı aşama. Aradaki %38,6'lık fark, koşular arası yayılımın (%9,4) çok üstünde olduğu için

gerçek bir farktır.
Doğruluk korunuyor: `prepared` 0,7677 / stres F1 0,5741, `native` 0,7674 / 0,5732 — pencere başına indirgemenin kenar etkisi ölçülemez düzeyde.
Bant genişliği bedeli: 500 ms'lik bir tick'te ham hızda 11 536 bayt, 64 Hz'e indirilmiş halde 1 792 bayt taşınır. İndirgemeyi sunucuda yapmak ağdan **6,4 kat** fazla veri geçirmek demektir. Uç/sis yerleşim kararının sayısal dayanağı budur.

5.2.3 Açıklayıcıların istemci makinesindeki CPU maliyeti

Yapılandırma	ortalama	p50	p95	p99	koşu arası yayılım
LIME 500 örnek / 20 dilim / kw 0,5	772,9	710,2	1129,9	1161,2	%7,4
LIME 500 örnek / **5 dilim** / kw 0,25	796,7	677,6	1203,5	1233,4	%69,2

Tablo 5.5: S11, 16 pencere, 3 tekrar, tek iplik, nedensel normalizasyon (ms). Kaynak: `02_benchmark/benchmark_xai_cpu.json`

Bu dosya hakkında iki not

1. Bu ölçüm **istemci makinesinde** yapıldı, sis düğümünde değil. Makalenin süre tablosu `04_fog/`'dan gelmeli.
2. Dosyadaki "`lime_ucuz`" kaydı, *sunucudaki* `lime_cheap` (250 örnek) ile aynı şey **değildir**: burada 500 örnek / 5 dilim / kw 0,25 kullanılmış. `results/README.md`'de bu koşuya "LIME 250 örnek p50 490 ms" denmesi **hatalıdır**; dosyada öyle bir satır yok. Bkz. Bölüm 12.

5.3 03_loopback — istemci ve sunucu ayrı süreçte, aynı makinede

Amaç. Yedi metriğin tamamını ilk kez ölçmek. TCP var, gerçek ağ yok.

Metrik	ortalama	medyan	p95	p99
latency	0,909	0,809	1,466	2,707
queuing delay	0,062	0,045	0,085	0,622
execution time	8,273	7,179	10,524	11,268
response time	9,884	8,821	12,698	13,927

Tablo 5.6: Tek istemci, gerçek zamanlı, S11'in ilk 10 dakikası, 104 ölçülen pencere (ms). Jitter 3,529 ms. Bant genişliği 184,8 kbps gönderilen + 2,3 kbps alınan = **187,2 kbps**. Sunucu içi aşama dökümü (ortalama): resampling 2,211, preprocessing 0,449, tensor_prep 0,324, inference 4,648, postprocessing 0,135. Saat belirsizliği ±0,129 ms. Kaynak: `03_loopback/fog_realtime_1user.json`

İkinci koşu (`fog_full_queue.json`). İki istemci, gerçek zaman *kapalı* (istemci mümkün olduğunca hızlı gönderir), 2123 ölçülen pencere. Amacı kuyruk gecikmesinin gerçekten ölçüldüğünü göstermek: queuing p99 **5,504 ms** (ortalama 0,092, p50 0,032). Açık kuyruk eklenmeden önce bu değer 0,37 ms'ydi.

Doğrulama

Tek süreçli `native` yol ile istemci–sunucu yolu **aynı veride birebir aynı** sınıflandırma sonucunu veriyor: doğruluk 0,8222, stres F1 0,6944, TN=743 FP=184 FN=29 TP=242. Bu, ağ katmanının

hesabı bozmadığının kanıtıdır.

Uyarı

`fog_full_queue.json`'daki bant genişliği (84 428 kbps) **gerçek yükü yansıtmaz**, çünkü `realtime` kapalıydı. Aynı sebeple o koşudaki queuing ve jitter da gerçek dağıtımı temsil etmez; koşunun amacı yalnızca kuyruk metriğinin çalıştığını göstermekti.

5.4 05_yerlesim — ortak yerleşim (uç) ve sis karşılaştırması

Kurulum. 3 tur dönüşümlü, S11, 600 tick, XAI yok, aynı denek/tick/süre; değişen tek şey sunucunun yeri.

Metrik (p50)	uç (ortak yerleşim)	yayılım	sis (LAN)	yayılım
latency	0,928	%7,0	4,910	%44,7
queuing	0,048	%7,3	0,028	%8,3
execution	**6,344**	%15,4	**3,372**	%6,3
response	7,880	%13,8	10,151	%22,9
response p95	10,360	%31,0	74,355	%112,1
jitter	2,221	%70,7	22,600	%91,2
bant genişliği	187,17 kbps		187,11 kbps	

Tablo 5.7: Üç tekrarın medyanlarının medyanı (ms). Kaynak: `05_yerlesim/deployments_edge_vs_fog.json`

Ne anlama geliyor

Sis düğümü hesabı **daha hızlı** yapıyor (3,37'ye karşı 6,34 ms): ortak yerleşimde istemci ve sunucu aynı işlemciyi paylaşıyor. Buna karşılık sis, Wi-Fi gecikmesini ödüyor ve yanıt süresi yine de daha yüksek çıkıyor (10,15'e karşı 7,88 ms). p95 ve jitter'daki büyük yayılım Wi-Fi'ın düzensizliğidir.

5.5 12_profil — aşama profilinin dört kaynaktan derlemesi

Amaç. Aynı aşamanın değeri kaynağa göre değişiyor ve sebepleri farklı; dağılmış hâlde bırakılırsa makalede yanlış satır alıntılanır.

Aşama (p50, ms)	tek süreç	tekrarlı native	tekrarlı prepared	sis uçtan uca
resampling	1,378	1,573 (%3,3)	—	1,044
preprocessing	0,218	0,252 (%3,5)	0,580 (%7,9)	0,172
tensor_prep	0,141	0,149 (%2,7)	0,135 (%5,7)	0,114
inference	2,682	3,039 (%8,9)	2,969 (%9,1)	1,948
postprocessing	0,086	0,087 (%3,0)	0,085 (%5,5)	0,046
toplam	4,689	**5,502 (%8,1)**	3,971 (%9,4)	**3,530**

Koşul: tek süreç = ağ yok, 2123 pencere, tek koşu · tekrarlı = 4 tur dönüşümlü, **yayılım taşıyan tek sütun** · sis = istemci–sunucu ayrı, gerçek ağ, 44 pencere

Tablo 5.8: Kaynak: `12_profil/profile_table_b1.json`. Üreten: `python -m tools.profile_table`

Hangi sütunu ne zaman kullanmalı

Belirsizlik/hata payı gerektiğinde **tekrarlı native** sütunu kullanılmalı — yayılım taşıyan tek sütun odur. Sis sütunu düşük çünkü o makine daha hızlı; tek süreç sütunu istemci makinesinde ölçüldü.

Bölüm 6

Deneyler II: Açıklamanın Maliyeti

6.1 06_xai_tarama — açıklayıcı hiperparametre taraması

Amaç. Her açıklayıcının hangi ayarla çalıştırılacağına karar vermek. *OFAT* (one-factor-at-a-time): her aşamada tek parametre değiştirilir.

Kurulum. Colab, GPU, 40 pencere, **nedensel** normalizasyon (yani sunucunun gördüğü girdi uzayı), seed sabit, ısınma var.

Yapılandırma	örnek	dilim	kw	gecikme p50	faithfulness	sensitivity
Aşama 1 — örnek sayısı						
250 örnek	250	10	0,25	76,0	0,2219	0,0274
500 örnek	500	10	0,25	102,4	0,2302	0,0266
1000 örnek	1000	10	0,25	213,8	0,2333	0,0264
Aşama 2 — bölge sayısı						
5 dilim	500	5	0,25	67,2	0,2180	0,0217
10 dilim	500	10	0,25	125,4	0,2302	0,0266
20 dilim	500	20	0,25	115,8	0,2375	0,0357
Aşama 3 — kernel genişliği						
kw 0,125	500	20	0,125	91,3	0,1887	0,0147
kw 0,25	500	20	0,25	138,1	0,2375	0,0357
kw 0,5	500	20	**0,5**	122,7	**0,2390**	0,0365

Tablo 6.1: LIME OFAT taraması (GPU, ms). *self-instability* dokuz yapılandırmanın hepsinde 0,0000 — LIME seed sabitken tam belirlenimci. Kaynak: `06_xai_tarama/xai_sweep_lime_causal.csv`

Yapılandırma	arka plan	nsamples	gecikme p50	faithfulness	sensitivity
Aşama 1 — arka plan boyutu					
bg 10	10	200	185,9	0,2038	0,1119
bg 50	50	200	192,9	0,2212	0,1003
bg 100	100	200	191,5	0,2243	0,1011
Aşama 2 — nsamples					
nsamples 50	100	**50**	**45,5**	0,2214	0,1083
nsamples 200	100	200	193,5	0,2243	0,1011
nsamples 400	100	400	379,5	0,2246	0,1032
Aşama 3 — arka plan seçim yöntemi					
random	100	200	190,0	0,2243	0,1011
kmeans	100	200	192,0	**0,2302**	0,1184

Tablo 6.2: SHAP (GradientExplainer) OFAT taraması (GPU, ms). Kaynak: `06_xai_tarama/xai_sweep_shap_causal.csv`

Taramadan çıkan dört ders

1. **LIME'da örnek sayısını 250'den 1000'e çıkarmak faithfulness'a 0,011 katıyor ama gecikmeyi 2,8 katına çıkarıyor.** Doyum noktası 500 civarında.
2. **SHAP'ta `nsamples` 50'den 400'e çıkarmak faithfulness'ı neredeyse hiç değiştirmiyor** (0,2214 → 0,2246) ama gecikmeyi 8,3 katına çıkarıyor. Ucuz nokta bedava sayılır.
3. **Arka planı 10'dan 100'e çıkarmak faithfulness'ı iyileştiriyor** (0,2038 → 0,2243), maliyeti değiştirmiyor.
4. **İki yöntemin sensitivity'si açık ara farklı:** LIME 0,015–0,037, SHAP 0,100–0,118. Bu ayrım bütün taramada tutarlı.

Seçilen yapılandırmalar (`checkpoints/xai_config.json`):

Ad	Yöntem	Yapılandırma
`lime`	LIME	500 örnek, 20 dilim × 14 kanal = 280 bölge, kw 0,5, seed 0
`lime_cheap`	LIME	aynı, **250 örnek**
`shap`	SHAP-GE	arka plan 100 (kmeans, S2–S5'ten), nsamples 200, seed 0
`shap_cheap`	SHAP-GE	aynı, **nsamples 50**

Seçim elle yapıldı ve gerekçesi kayıtta

Otomatik seçim yalnızca faithfulness'a baktığı için **kullanılmadı**. `xai_config.json`'daki not: "Eşit maliyette LIME daha sadık (0,2390 > 0,2302) ve 3,2 kat daha kararlı (0,036 < 0,118)." Ayrıca sızıntılı uzayda ölçülen önceki koşuda XAI *daha iyi* görünüyordu — nedensel uzay gerçek dağıtım koşuludur.

Süre değerleri GPU'ya aittir. Makalenin süre tablosu `04_fog/`'dan gelmeli; kalite metrikleri ise donanımdan bağımsız olduğu için buradan alınabilir.

6.2 04_fog — makalenin ana tablosu

Kurulum. Sunucu ikinci bir bilgisayarda: 192.168.1.14, Intel 12 çekirdek @ 2592 MHz, 16 GB, Python 3.14.7, PyTorch 2.14.0+cpu, tek iplik. İstemci bu makinede, Wi-Fi üzerinden. S11, tek istemci, 600 tick

(5 dk), **44 ölçülen pencere**, saat farkı 60 saniyede bir yeniden ölçülüyor.

Açıklayıcı	xai	execution	response p50	p95	latency p50	CPU	RSS tepe
yok	—	3,59	**10,62**	42,23	4,46	%0,0	210,4
shap_cheap	395,03	400,18	**442,61**	574,01	4,32	%5,0	422,3
lime_cheap	590,09	594,18	**645,22**	802,47	5,27	%10,8	442,4
lime	1188,79	1192,88	**1227,54**	1407,12	4,95	%22,8	449,2
shap	1422,28	1427,43	**1456,17**	1548,50	4,52	%25,9	464,5

Tablo 6.3: Sis düğümü, beş durum (ms; CPU tek çekirdeğe göre yüzde; RSS MB). "xai" ve "execution" ortalama, response medyan ve p95. Kaynak: `04_fog/fog_xai_*.json`

Açıklayıcı	resampling	preprocessing	tensor	inference	postproc	xai
yok	1,052	0,170	0,113	2,008	0,047	—
shap_cheap	1,120	0,190	0,119	3,458	0,054	395,03
lime_cheap	1,091	0,170	0,114	2,460	0,048	590,09
lime	1,073	0,172	0,114	2,488	0,047	1188,79
shap	1,051	0,171	0,114	3,569	0,047	1422,28

Tablo 6.4: Aynı koşuların sunucu içi aşama dökümü (ortalama, ms). Açıklama dışındaki bütün aşamalar sabit — açıklayıcı seçimi onları etkilemiyor.

Bu tablodan çıkan dört sonuç

1. Açıklama, sistemin baskın maliyetidir. Pencere başına toplam sunucu hesabı, açıklayıcı kapalıyken 3,39 ms; açıklayıcı açıkken:

	toplam hesap	**taban çizgiye oran**
shap_cheap	399,98 ms	**118**×
lime_cheap	593,98 ms	**175**×
lime	1192,68 ms	**352**×
shap	1427,23 ms	**421**×

Yanıt süresi cinsinden: 10,6 ms'den 1456 ms'ye, yani **137 kat**.
2. Ucuz çalışma noktaları 1 s/pencere sınırının altında kalıyor (SHAP 443 ms, LIME 645 msmedyan); pahalı olanlar p95'te aşıyor.
3. Bellek 210 MB'dan 464 MB'a çıkıyor. 2 GB'lık bir Raspberry Pi'de bu sıkışır; en az 4 GB gerekir.
4. Latency hepsinde 4,3–5,3 ms bandında, yani açıklayıcı seçimi ağı etkilemiyor. Bu beklenen davranıştır ve ölçümün *tutarlılık kontrolü* olarak işe yarar.

Bu tablonun üç sınırı

1. **44 pencere azdır.** Ortalama karşılaştırması geçerli, ama p95/p99 istatistiksel olarak zayıf. Güçlü p95 yalnızca 1030 pencerelik seçici koşudan (Bölüm 7.2) gelir.
2. **Sınıflandırma sayıları anlamsızdır.** Bu koşular S11'in ilk beş dakikasını kapsıyor ve o aralıkta hiç stres etiketi yok (42 pencerenin hepsi stres-olmayan, doğruluk 1,00). Sınıflandırma başarımı bu tablodan okunmamalı.
3. **Kayıt boşluğu (düzeltildi).** Bu koşular yapıldığında sunucu, hangi açıklayıcının kullanıldığını cevaba yazmıyordu; bilgi yalnızca dosya adındaydı. Sonradan eklendi — 09_secici_xai koşula-

rında `server.xai` alanı (yöntem, eşik, ısınma süresi, tam yapılandırma) mevcut.

6.3 07_edge_xai — aynı beş durum, ortak yerleşimde

Kurulum. 04_fog ile *birebir* eşleşen koşular: aynı denek (S11), aynı süre (600 tick), aynı yöntem. Değişen tek şey sunucunun yeri: burada istemciyle aynı makinede (Intel 20 çekirdek, 16 GB, Python 3.11.9, PyTorch 2.8.0+cpu).

Açıklayıcı	xai (ort.)	execution p50	response p50	RSS tepe
yok	—	5,81	**7,31**	187,0
`shap_cheap`	417,13	460,44	**462,01**	493,3
`lime_cheap`	389,64	351,49	**352,81**	430,6
`lime`	830,93	688,54	**690,05**	448,0
`shap`	1223,60	1152,43	**1154,45**	523,3

Tablo 6.5: Ortak yerleşim (ms, MB). Loopback olduğu için response ≈ execution; aradaki fark 04_fog'da gerçek ağ maliyetidir. Kaynak: `07_edge_xai/edge_xai_*.json`

Bu tablo 04_fog ile doğrudan kıyaslanamaz

Sıralama yönteme göre değişiyor:
- LIME'da uç hızlı (390'a karşı 590 ms)
- SHAP'ın ucuz noktasında sis hızlı (395'e karşı 417 ms)
- Tabanda sis hızlı (3,5'e karşı 5,8 ms)

Donanım farkı böyle davranmaz. Sebep **yazılım yığını farkı**: uçta PyTorch 2.8.0 / Python 3.11, siste 2.14.0 / Python 3.14. LIME toplu ileri geçiş, SHAP gradyan hesabı yapıyor; iki farklı kod yolu iki sürümden farklı etkileniyor. İkisi de her koşunun `server_hardware` alanında kayıtlı.
Makalede ya bu cümle yazılır, ya da sis düğümü Docker'la (python:3.11) yeniden koşulup yığın eşitlenir. Olduğu gibi "sis daha yavaş" diye sunulamaz.

6.4 13_bulut — bulut yerleşimi

Kurulum. Sunucu bir DigitalOcean VPS'te (167.71.43.64, "DO-Regular" CPU, **1 çekirdek**, 2 GB RAM, Python 3.11.16, PyTorch 2.14.0), Docker konteynerinde. İstemci bu makinede, internet üzerinden.

Açıklayıcı	xai	execution	response p50	p95	latency p50
yok	—	4,08	**63,52**	154,54	39,84
`shap_cheap`	337,37	341,66	**429,84**	508,98	60,48
`lime_cheap`	360,95	365,10	**463,26**	608,32	53,12
`lime`	782,37	786,66	**870,09**	1091,77	53,97
`shap`	1220,05	1224,39	**1300,46**	1715,85	54,76

Tablo 6.6: Bulut, tek istemci (ms). Taban satırı 3 tur dönüşümlü tam kayıttan, XAI satırları 600 tick / 44 pencereden. Kaynak: `13_bulut/cloud_taban_3tur.json`, `cloud_xai_*.json`

Açıklayıcı	uç (ortak)	sis (LAN)	bulut (internet)	bulut/uç
yok	7,31	10,62	**63,52**	**8,69**×
`shap_cheap`	462,01	442,61	429,84	0,93×
`lime_cheap`	352,81	645,22	463,26	1,31×
`lime`	690,05	1227,54	870,09	1,26×
`shap`	1154,45	1456,17	1300,46	**1,13**×

Tablo 6.7: Üç yerleşim yan yana (medyan yanıt süresi, ms).

Raporun en çarpıcı sonucu

Açıklama açıkken yerleşim farkı önemsizleşiyor. XAI'siz durumda bulut, uçtan **8,7 kat** yavaş; `shap` ile yalnızca **1,13 kat** — çünkü 50 ms'lik ağ farkı 1200 ms'lik açıklamanın yanında kayboluyor.
Fark ağdan geliyor, hesaptan değil. Latency uçta ≈1 ms, siste 4,3–5,3 ms, bulutta **40–60 ms**. Hesap tarafında bulut VPS'i sis düğümünden yavaş *değil*; `shap_cheap`'te hatta biraz hızlı (337'ye karşı 395 ms) — ve bunu tek çekirdekle yapıyor.
Bu, referans makalenin "fog, cloud'dan üstündür" sonucunu **XAI yükü altında geçersiz kılıyor**.

Bulutta 4 istemci yükü (`cloud_load_4.json`): **4120 ölçülen pencere**, 87 dakika. Queuing ortalama 4,74 ms (p95 16,08), execution 4,81 ms, response ortalama 67,48 ms (p50 62,06, p95 117,81). Sis düğümündeki XAI'siz 4 istemci koşusuna (queuing p95 12,50 ms) yakın — beklenen, çünkü ikisinde de açıklama kapalı. Sınıflandırma: 2416 etiketli pencere, doğruluk 0,9520, stres F1 0,8807.

Uyarı

Bu sütunlar **farklı yazılım yığınlarında** ölçüldü: bulut Docker'da python:3.11 + torch 2.14, sis düğümü python 3.14 + torch 2.14, uç python 3.11 + torch 2.8. Yerleşim karşılaştırması bu farkı içerir.
Ayrıca bulut koşularında saat belirsizliği ±**24–51 ms** — latency sayıları bu hata payıyla okunmalı. Execution, queuing (sunucu saati) ve response, jitter (istemci saati) bundan etkilenmez.

6.5 11_pareto — kalite ile gecikmenin dengesi

Kurulum. Kalite *bu makinede* ölçüldü (donanımdan bağımsız), gecikme `04_fog/`'dan alındı (dağıtım donanımı). S11, 40 pencere; sensitivity 10 pencere × 3 deneme.

Nokta	gecikme	faithfulness ± SEM	monotonicity	sensitivity ± SEM
`shap_cheap`	395 ms	0,2499 ± 0,0426	0,5650	0,1228 ± 0,0032
`lime_cheap`	590 ms	0,2092 ± 0,0345	0,6038	**0,0489** ± 0,0016
`lime`	1189 ms	0,2324 ± 0,0373	0,5788	**0,0446** ± 0,0018
`shap`	1422 ms	0,2632 ± 0,0445	0,5488	0,1282 ± 0,0046

Tablo 6.8: Dört çalışma noktası. Kaynak: `11_pareto/pareto_c5.json`

Üç sonuç

1. Faithfulness'ta hiçbir nokta diğerini yenmiyor. 40 pencerede standart hata ≈0,04; dört nokta 0,209–0,263 aralığında, yani farkların hepsi hata payının içinde. Seçim faithfulness'a göre

yapılamaz. Colab taramasının 0,003'lük farklarla kazanan seçmesi bu yüzden yanlıştı.
2. Ayrışma sensitivity'de ve kesin. LIME 0,045–0,049, SHAP 0,123–0,128. Fark hata payının **15 katı**. LIME **2,7 kat daha kararlı** açıklama üretiyor.
3. Gerçek denge: SHAP 3 kat ucuz, LIME 2,7 kat kararlı, sadakat eşit. Hangi yöntemin seçileceği uygulamaya bağlıdır — "en iyi" yoktur.

Tek örnek başına maliyet tersine dönüyor

LIME örnek başına 2,4 ms ($590/250$), SHAP örnek başına 7,9 ms ($395/50$). Yani *tek örnek* düzeyinde SHAP daha pahalı — literatürle uyumlu. Ama SHAP çok daha az örnekle yetindiği için **toplamda ucuz**. Bu ayrım makalede yazılmalı, yoksa çelişkili görünür.

Bölüm 7

Deneyler III: Seçici Açıklama

7.1 08_kalibrasyon — seçici açıklamanın ön koşulu

Soru. Modelin `confidence` değeri gerçek doğruluk olasılığını temsil ediyor mu? Etmiyorsa güven eşiği anlamsız olur.

Kurulum. S11 ve S14, sunucunun gördüğü girdi uzayında (native yeniden örnekleme + nedensel z-score), 15 kutu.

	S11	**S14**
pencere sayısı	604	594
doğruluk	**0,9520**	**0,6902**
ortalama güven	0,9093	0,9085
fazla özgüven	−0,0426	+**0,2182**
ECE	**0,0469** (iyi)	**0,2265** (bozuk)
MCE	0,2523	0,5224
Brier	0,0379	0,2657
NLL	0,1381	0,8984
hata sayısı	29	184

Tablo 7.1: İki tutulmuş denek taban tabana zıt. Kaynak: `08_kalibrasyon/calibration_d1.json`

Ana gözlem

Model **ikisine de aynı güvenle yaklaşıyor** (0,909 ve 0,909) ama birinde %95, diğerinde %69 haklı. S14'te model **kendinden emin biçimde yanılıyor** — bu, seçici açıklama açısından en kötü durumdur.

7.1.1 Sıcaklık ölçekleme çalışmadı

Denek	**Sıcaklık nereden**	T	**ECE önce**	**ECE sonra**
S11	S14'ten	3,484	0,0469	**0,2557**
S14	S11'den	0,578	0,2265	**0,2755**

Tablo 7.2: Çapraz denek sıcaklık ölçekleme her iki yönde de *kötüleştirdi*.

Bu bir hata değil, bulgudur

Bozukluk **zıt yönlerde**: S11 az güvenli (fazla özgüven −0,04), S14 çok fazla güvenli (+0,22). Tek bir T parametresi ikisini birden düzeltemez. Bu, makalede raporlanacak bir sonuçtur, gizlenecek bir başarısızlık değil.

7.1.2 Aday eşikler (düzeltmesiz)

	S11 (iyi kalibre)			**S14** (bozuk kalibre)		
eşik	**açıklanan**	**hata yakalama**	**tetik isabeti**	**açıklanan**	**hata yakalama**	**tetik isabeti**
0,60	%6,3	0,379	0,289	%1,7	0,027	0,500
0,70	%10,4	0,621	0,286	%4,4	0,060	0,423
0,80	**%14,4**	**0,897**	0,299	%11,8	0,120	0,314
0,90	%21,4	1,000	0,225	%29,3	0,288	0,305

Tablo 7.3: Eşik taraması. "Açıklanan" = açıklama tetiklenen pencere oranı, "hata yakalama" = yanlış sınıflanan pencerelerin kaçında açıklama tetiklendi.

Makaleye giren ayrım — iki iddia ayrı yazılmalı

Maliyet tasarrufu her iki denekte gerçekleşiyor. Eşik 0,80'de açıklama oranı %12–14, yani her pencereyi açıklamaya göre ≈%86 daha az hesap. Bu, kalibrasyondan *bağımsızdır*.
Fayda ise kalibrasyona bağlıdır. İyi kalibre denekte hataların %89,7'si yakalanıyor, kötü kalibre denekte yalnızca %12'si.

Güven dağılımı. S11'in güven yüzdelikleri: p1 = 0,512, p5 = 0,578, p25 = 0,918, p50 = 0,961, p95 = 0,992. Yani **pencerelerin çeyreğinden fazlası 0,92'nin üzerinde** — düşük güven nadirdir ve seçici açıklamanın kazancı tam buradan gelir.

7.2 09_secici_xai — ana sonuç

7.2.1 Tam kayıt koşusu

Kurulum. S11'in **tam kaydı** (10 466 tick, 87 dakika), sis düğümü, `shap_cheap`, eşik **0,80**. **1030 ölçülen pencere** — diğer bütün sis koşularının 23 katı; p95 ve p99 burada gerçekten anlamlı. Saat farkı 88 kez yeniden ölçüldü (kayma −7,03 ppm).

Durum	**pencere**	**response ort.**	**p50**	**p95**	**p99**
hiç açıklama (`04_fog/none`)	44	15,93	10,62	42,23	91,31
seçici (0,80)	**1030**	**80,13**	**10,95**	**445,70**	517,03
hep açıkla (`04_fog/shap_cheap`)	44	450,54	442,61	574,01	599,39

Tablo 7.4: Üç durum yan yana (ms). Kaynak: `09_secici_xai/fog_sel80_tam.json`

Ölçülen ana sonuç

Seçici açıklama, her pencereyi açıklamaya göre 5,6 kat ucuz (450,54/80,13).
- Açıklama oranı **%14,1** (145 / 1030 pencere)
- Açıklama başına 364,82 ms; pencere *başına* amorti edilmiş 51,36 ms

- Kalibrasyon tablosuna göre hataların **%89,7**'si hâlâ açıklanıyor
- **Medyan yanıt süresi 10,95 ms** — yani tipik pencere, açıklama hiç yokmuş gibi hızlı cevaplanıyor
- Sınıflandırma: 604 etiketli pencere, doğruluk 0,9520, stres kesinliği 1,000, duyarlılık 0,7868, F1 0,8807 (TN 468, FP 0, FN 29, TP 107)

Kuyrukta birikme YOK

Queuing ortalama 0,031 ms, p99 0,057 ms. Patlamalı yükün (pencerelerin %86'sı 3,6 ms, %14'ü ≈365 ms) kuyruğu bozması beklenebilirdi; *bozmadı*. Sebebi basit: hop bütçesi 5000 ms ve en pahalı pencere bile 365 ms sürüyor, yani bir sonraki pencere gelmeden iş bitiyor. Bu, karışım modelinin kaçırabileceği bir etkileşim etkisi olmadığını gösterir.

7.2.2 Projeksiyon ölçümle doğrulandı

`selective_xai_sweep_d2.json`, kalibrasyonun açıklama oranını `04_fog`'un maliyetiyle birleştiren bir **karışım modelidir**:

$$\text{ortalama} = (1-p)\cdot \text{taban} + p \cdot \text{açıklamalı}$$

Sadece eşik 0,0 ve 1,0 satırları ölçülmüştür; aradakiler türetilmiştir. Ana koşu bunu test etti:

	tahmin	ölçüm	hata
açıklama oranı	%14,4	%14,1	**%2,3**
ortalama response	78,53 ms	80,13 ms	**%2,0**
p95 response	487,25 ms	445,70 ms	%8,5

Tablo 7.5: Karışım modeli ile gerçek ölçümün karşılaştırması.

Ortalamadaki %2 beklenen, çünkü karışımın ortalaması matematiksel olarak kesindir. p95'teki %8,5, özet çeyrekliklerden ara değerle tahmin edildiği için baştan "yaklaşık" diye işaretliydi. **Sonuç:** dosyadaki diğer satırlar (diğer açıklayıcılar, S14) güvenle projeksiyon olarak sunulabilir.

Açıklayıcı	Denek	eşik	açıklama oranı	hata yakalama	response ort.	p95
`shap_cheap`	S11	0,0	0,000	0,000	15,93	42,23
`shap_cheap`	S11	0,7	0,104	0,621	61,26	448,64
`shap_cheap`	S11	**0,8**	0,144	0,897	**78,53**	487,25
`shap_cheap`	S11	0,9	0,214	1,000	108,75	520,25
`shap_cheap`	S11	1,0	1,000	1,000	450,54	574,01
`shap_cheap`	S14	0,8	0,118	0,120	67,15	464,72
`lime_cheap`	S11	0,8	0,144	0,897	109,18	698,64
`lime`	S11	0,8	0,144	0,897	193,88	1288,55
`shap`	S11	0,8	0,144	0,897	225,34	1487,54

Tablo 7.6: Seçici açıklama taramasından seçilmiş satırlar (ms). Tam tablo 48 satır: 4 açıklayıcı × 2 denek × 6 eşik. Kaynak: `09_secici_xai/selective_xai_sweep_d2.json`

7.2.3 Eşiğin hiç tetiklenmediği üç koşu

`kisa_segment_sel{60,80,90}.json`: 600 tick'lik (5 dk) koşular, **üçünde de açıklama oranı %0**.

Bu bir hata değil, kayıt içi durağansızlığın kanıtı

S11'in ilk 5 dakikasında en düşük güven **0,929**; en yüksek eşik (0,90) bile hiçbir pencerenin altına inmiyor. Düşük güvenli ilk pencere **#100**, yani kaydın 8. dakikasında. Kayıttaki 87 düşük güvenli pencerenin hepsi ilk 400 pencere içinde; sonrası tekrar yüksek güvene dönüyor.
Seçici XAI deneyi bu yüzden **tam kayıt üzerinde** koşuldu. Bu, sistem deneylerinde "5 dakikalık örnek yeterlidir" varsayımının neden tehlikeli olduğunun somut kanıtıdır.

Dosya adları yanıltıcı

`sel60` gerçekte eşik **0,70** ile koşuldu. Etiket kayması koşu sırasında oldu. Gerçek eşikler sunucu künyesinde: `server.xai.threshold` = 0,7 / 0,8 / 0,9. **Eşik 0,60 hiç koşulmadı.** Dosya adına değil künyeye bakılmalı — künyenin kayda geçirilmesinin gerekçesi tam olarak budur.

7.3 10_yuk — çok istemcili yük

Kurulum. 1 / 2 / 4 istemci, sis düğümü, S11+S14 döngüsel, 600 tick. Tek istemci satırları `04_fog/`'dan.

Durum	istemci	queuing ort.	q p95	execution	response ort.	p95
XAI yok	1	0,030	0,035	3,59	15,93	42,23
XAI yok	2	1,607	4,514	3,57	24,31	107,78
XAI yok	4	4,555	12,505	3,69	28,50	120,27
hep açıkla	1	0,032	0,052	400,18	450,54	574,01
hep açıkla	2	**160,12**	381,58	371,06	601,30	853,58
hep açıkla	4	**437,25**	**1125,11**	370,75	984,94	1643,07

Tablo 7.7: Yük altında yedi metrik (`shap_cheap`, ms). Kaynak: `10_yuk/fog_load_*.json` + `04_fog/`

Mekanizma ortalama doluluk değil, eşzamanlı patlama

4 istemcide teorik doluluk yalnızca %32 iken ölçülen kuyruk p95 1125 ms. Sebep: istemciler birlikte başlatıldığı için **faz hizalı** kalıyor, pencerelerini aynı anda üretiyor ve tek worker'da açıklamalar arka arkaya diziliyor. Basit model tutuyor:

$$(N-1) \times \text{açıklama süresi} = 3 \times 370{,}75 = 1112\,ms \qquad \text{ölçülen p95} = 1125\,ms$$

XAI'siz tarafta da tutuyor: $3 \times 3{,}69 = 11{,}1$ vs. ölçülen 12,5 ms.

Durum	execution	kapasite (5000/exec)
XAI yok	3,59 ms	**1394** istemci
seçici (0,80)	55,26 ms	**90** istemci
hep açıkla	400,18 ms	**12** istemci
referans makale, tek worker		≈20 istemci

Tablo 7.8: Tek worker kapasitesi. Seçici açıklama kapasiteyi **7,5 kat** artırıyor.

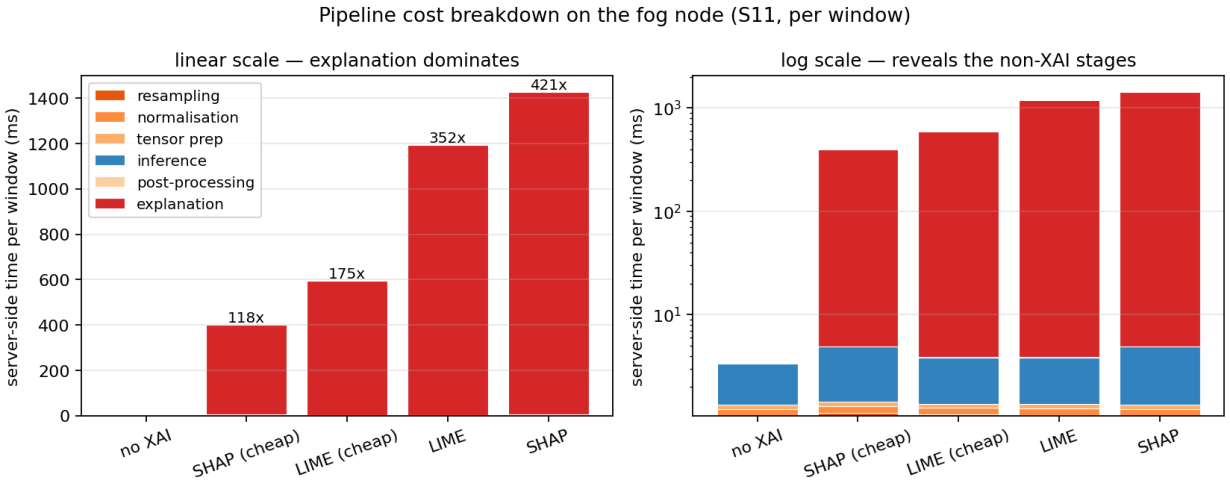
Uyarı

Bu bir **üst sınırdır**. Gerçek dağıtımda istemciler rastgele fazlarda başlar ve kuyruk bu kadar kötü olmaz. Raporda ve makalede belirtilmelidir.

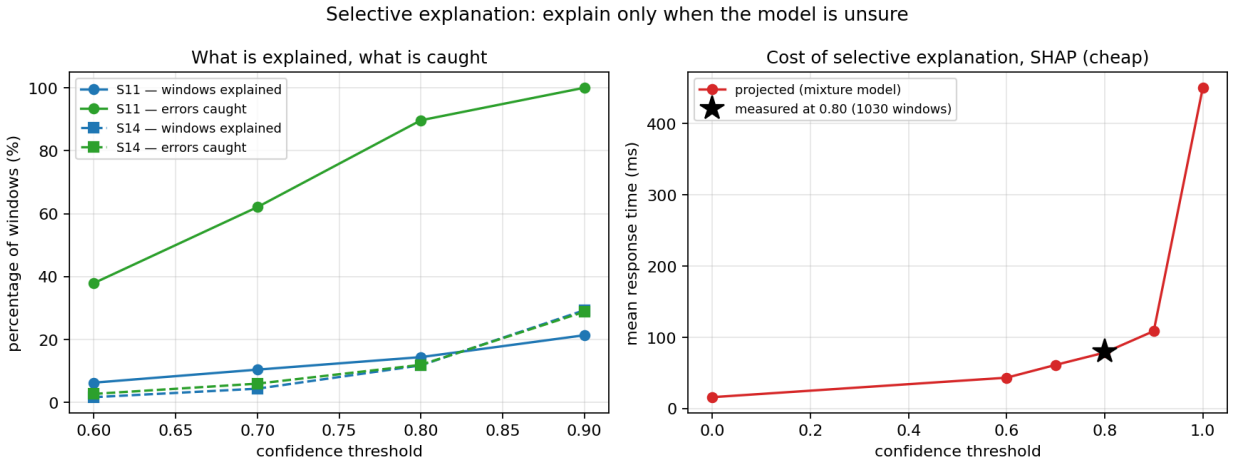
Bölüm 8

Şekiller ve Nasıl Okunmalı

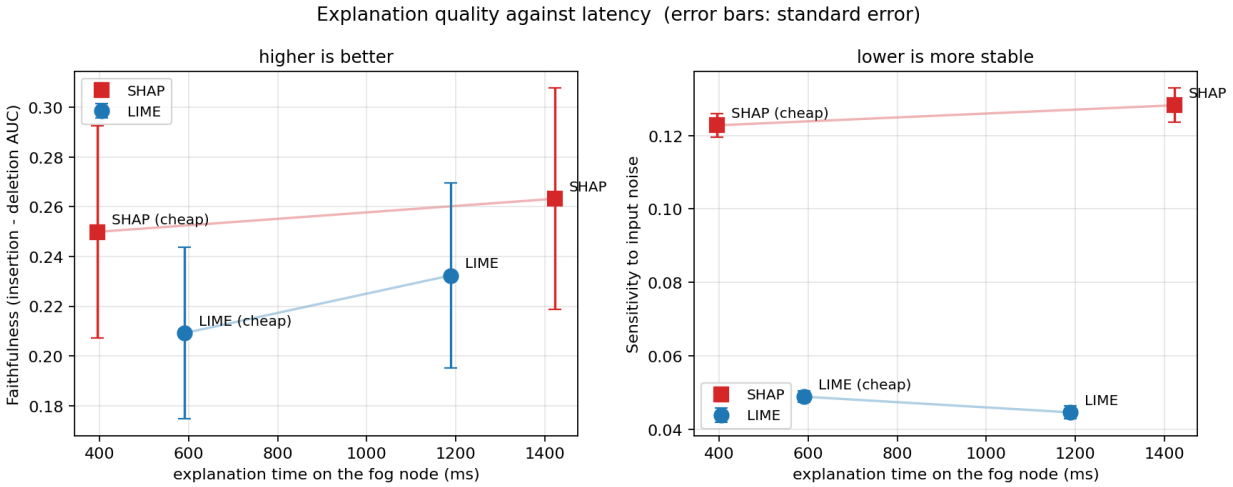
Altı şeklin hepsi `python -m tools.figures` ile JSON'lardan üretilir; elle sayı taşınmaz. Şekiller İngilizcedir (makale için hazır); burada Türkçe okunuşları verilmiştir.



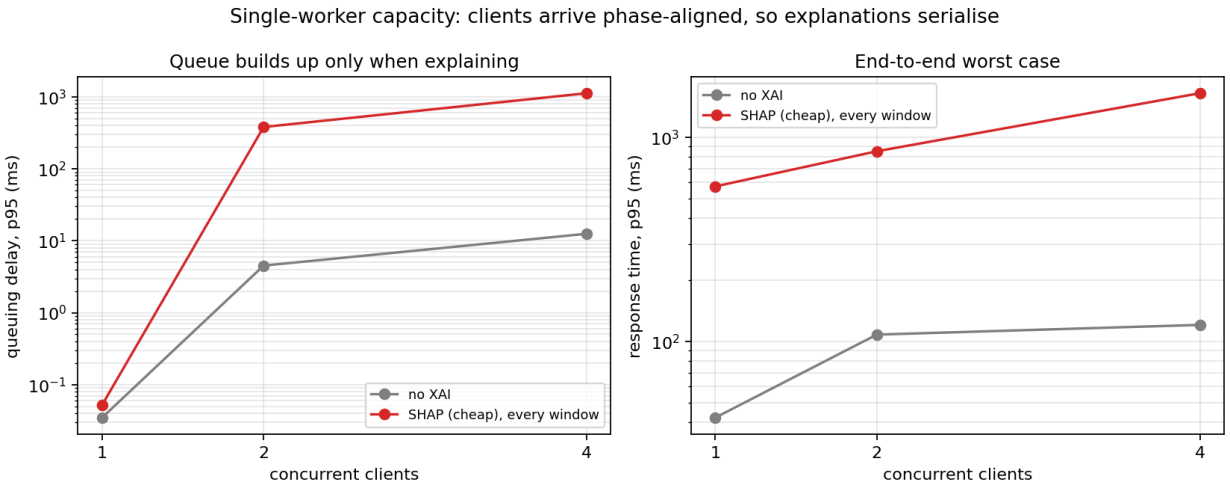
Şekil 8.1: Maliyet dökümü. Sol panel doğrusal ölçekte: açıklama (kırmızı) bütün grafiği kaplıyor, diğer aşamalar görünmüyor bile. Sağ panel logaritmik ölçekte: aynı veri, ama artık yeniden örnekleme (turuncu) ve çıkarım (mavi) seçilebiliyor. Çubukların üstündeki "118x" gibi etiketler, o yapılandırmanın toplam sunucu hesabının açıklayıcısız tabana (3,39 ms) oranıdır. *Bu şekil tek başına raporun ana bulgusunu taşır.*



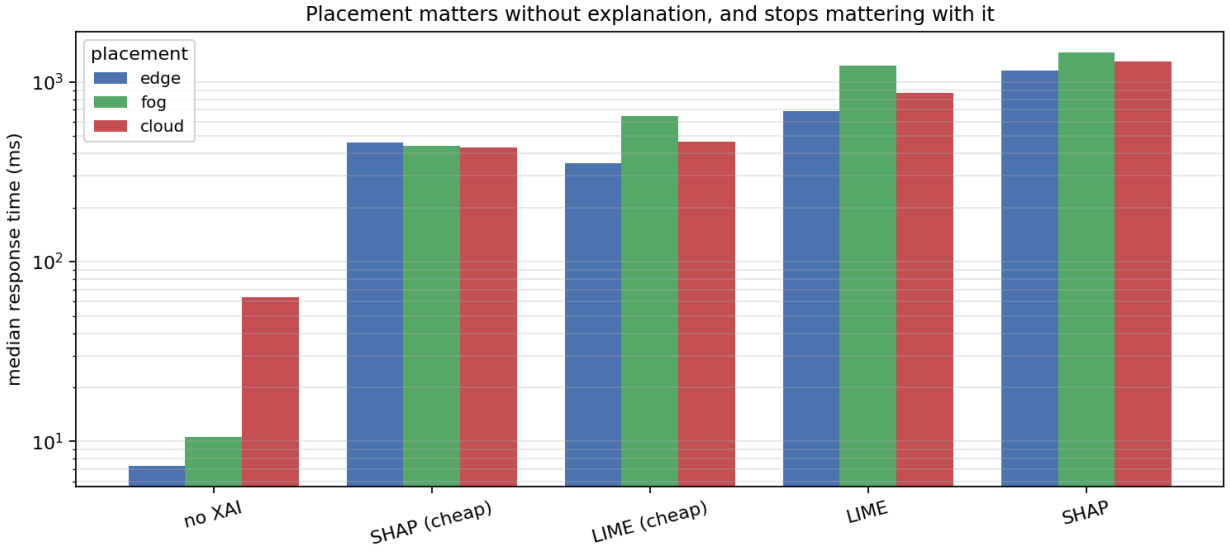
Şekil 8.2: Seçici açıklama. Sol panel: eşik yükseldikçe (yatay eksen) açıklanan pencere oranı (mavi) yavaş, yakalanan hata oranı (yeşil) hızlı artıyor — S11'de (düz çizgi) 0,80 eşiğinde %14 pencereyle hataların %90'ı yakalanıyor. S14'te (kesikli) aynı eğri çok daha düşük: kalibrasyon bozuk olduğu için hata yakalama %12'de kalıyor. Sağ panel: eşiğe karşı ortalama yanıt süresi. Kırmızı eğri karışım modelinin *projeksiyonu*, siyah yıldız **1030 pencerelik gerçek ölçüm** — ikisi üst üste düşüyor.



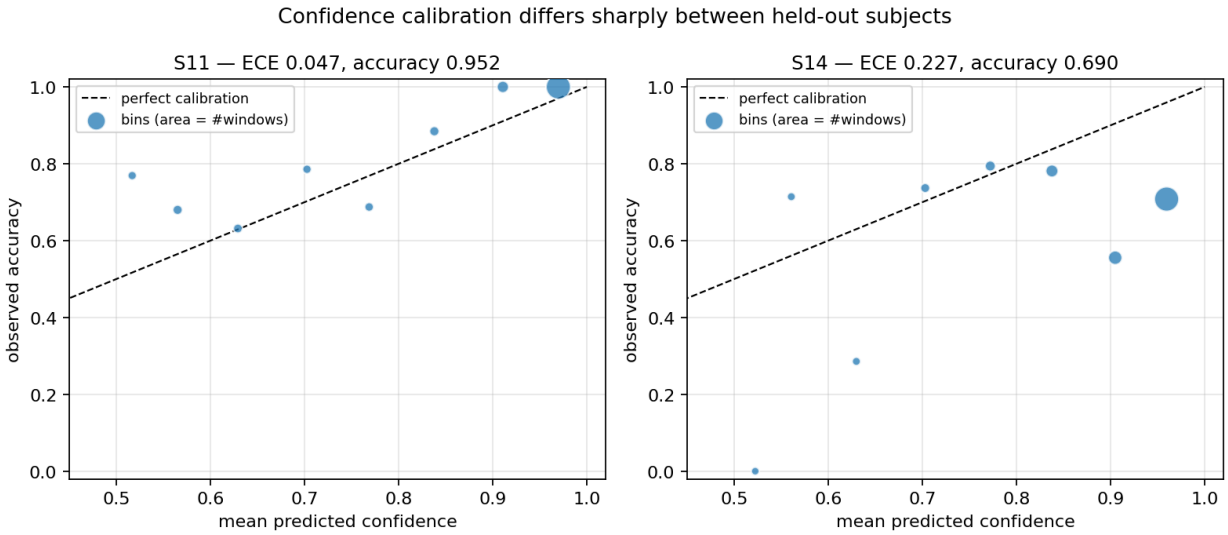
Şekil 8.3: Kalite–gecikme dengesi. Sol panel faithfulness: dört noktanın hata çubukları *örtüşüyor*, yani sadakat farkı yok. Sağ panel sensitivity: LIME (mavi, altta) ile SHAP (kırmızı, üstte) arasında açık ve hata payının çok üstünde bir ayrım var. *Karar faithfulness'a göre değil, kararlılık ve maliyete göre verilmeli.*



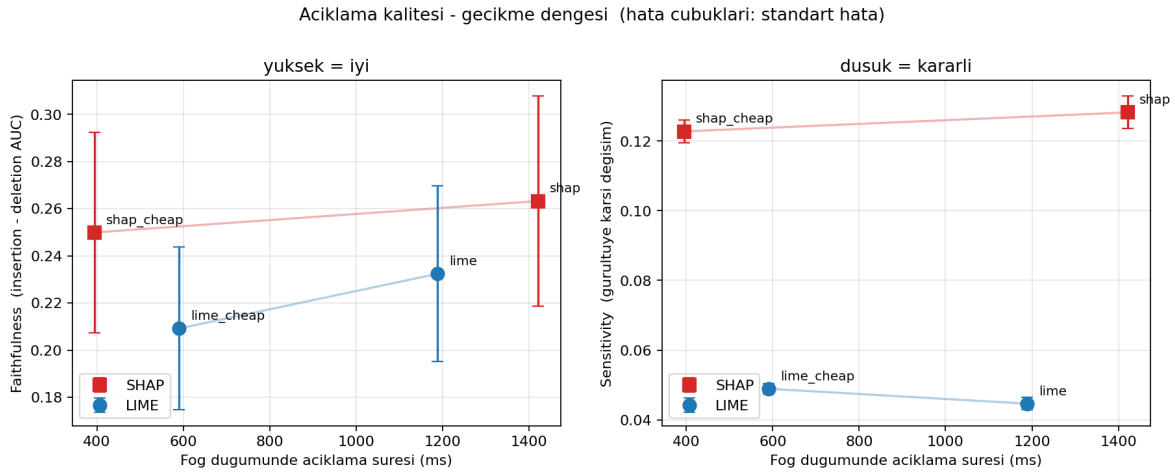
Şekil 8.4: **Yük altında kapasite** (her iki eksen logaritmik). Sol panel kuyruk gecikmesi p95: açıklama kapalıyken (gri) 4 istemcide 12,5 ms'de kalıyor, açıkken (kırmızı) 1125 ms'ye fırlıyor — iki büyüklük mertebesi fark. Sağ panel aynı etkinin uçtan uca yansıması. Eğrilerin doğrusal (log-log'da) olması $(N-1)\times$açıklama modelini doğruluyor.



Şekil 8.5: **Yerleşim kararı** (dikey eksen logaritmik). En soldaki grupta ("no XAI") üç çubuk arasında büyük fark var: uç 7 ms, sis 11 ms, bulut 64 ms. Sağa doğru gidildikçe, yani açıklama pahalılaştıkça, üç çubuk birbirine yaklaşıyor ve en sağdaki grupta ("SHAP") neredeyse eşitleniyor. *Yerleşim kararı, açıklama yükü altında anlamını yitiriyor.*



Şekil 8.6: Güven kalibrasyonu. Kesikli köşegen mükemmel kalibrasyonu gösterir; baloncuk alanı o kutudaki pencere sayısıdır. S11 (sol) noktaları köşegenin üstünde veya yakınında — model gerektiği kadar emin. S14 (sağ) en büyük baloncuk köşegenin *altında*: model %96 güvenle konuşuyor ama %71 haklı. İkisinin ortalama güveni aynı (0,909); fark tamamen doğrulukta.



Şekil 8.7: Pareto (ayrı üretim). Şekil 3 ile aynı veriyi gösterir; `tools/pareto.py` tarafından üretilen bağımsız sürümüdür.

Bölüm 9

Donanım Fizibilitesi: Model Bir Mikrodenetleyicide Çalışır mı?

`tools/mcu_feasibility.py` modelin bir MCU'da çalışıp çalışmayacağını *donanım olmadan* hesaplar: katman katman parametre, aktivasyon belleği, tepe RAM ihtiyacı ve MAC sayısı.

Katman	Şekil	Parametre	MAC	Aktivasyon
giriş	14×640	0	0	8 960
conv1	32×640	4 064	2 580 480	20 480
bn1	32×640	64	20 480	20 480
pool1	32×320	0	10 240	10 240
conv2	128×320	36 992	11 796 480	40 960
bn2	128×320	256	40 960	**40 960**
pool2	128×160	0	20 480	20 480
gap	128	0	20 480	128
fc1	64	8 256	8 192	64
bn_fc	64	128	64	64
fc2	2	130	128	2
TOPLAM		**49 890**	**14 497 984**	

Tablo 9.1: Katman katman döküm. Tepe aktivasyon **81 920 eleman** (`bn2` katmanında) — ikinci evrişim bloğu 128 kanal × 320 uzunluk ürettiği için darboğaz orada.

Veri tipi	Ağırlık (Flash)	Tepe RAM	Giriş tamponu
float32	194,9 KB	320,0 KB	35,0 KB
float16	97,4 KB	160,0 KB	17,5 KB
int8	48,7 KB	80,0 KB	8,8 KB

STM32	RAM	Flash	fp32	int8
F103 (M3)	20 KB	128 KB	sığmaz	sığmaz
F407 (M4)	192 KB	1 MB	sığmaz	**sığar**
F746 (M7)	320 KB	1 MB	sığmaz	**sığar**
H743 (M7)	1 MB	2 MB	**sığar**	**sığar**
U575 (M33)	786 KB	2 MB	**sığar**	**sığar**

Tablo 9.2: Bellek ihtiyacı ve STM32 ailelerine göre durum. Flash'ın %80'i, RAM'in %80'i sınırı alındı; kalanı program kodu, yığın ve tampon için.

Donanım	MAC/çevrim	tahmini süre	bütçe kullanımı
Cortex-M4 @168 MHz, float32 (CMSIS)	0,3	287,7 ms	%5,8
Cortex-M7 @480 MHz, float32 (CMSIS)	0,8	37,8 ms	%0,8
Cortex-M7 @480 MHz, int8 (CMSIS-NN)	2,5	12,1 ms	%0,2

Tablo 9.3: Hız tahmini. Bütçe 5000 ms (hop süresi).

Hız sorun değil, bellek sorun

En yavaş senaryo bile hop bütçesinin yirmide birini kullanıyor. MCU'da darboğaz kesinlikle **bellek**tir; int8 nicemleme F407/F746 için zorunludur.

Bu analiz kâğıt üstündedir

Cihaza indirmek için gereken iş: (1) ONNX'e aktarım + STM32Cube.AI ile C kodu, (2) int8 nicemleme ve doğruluk kaybının ölçümü, (3) ön işlemenin portu (700 Hz→64 Hz ve nedensel z-score), (4) cihaz üstü ölçüm (DWT çevrim sayacı, UART raporu), (5) veri beslemesi (SD kart replay ya da UART). Bu birkaç haftalık bir iştir ve donanım gerektirir.

Mimarideki yeri. STM32 bir mikrodenetleyicidir, işletim sistemi çalıştırmaz; sis taksonomisinde Raspberry Pi'nin yerini *alamaz*. Ama daha ilginç bir şeyin yerini alabilir: **ağın tamamının**. Çıkarım giyilebilir cihazın içinde yapılırsa uç/sis/bulut seçimi sorunu ortadan kalkar. *Not:* bu, açıklamayı cihaza taşımaz — bu raporun ana bulgusu göz önüne alındığında, XAI'yi bir MCU'ya sığdırmak ayrı ve çok daha zor bir problemdir.

Bölüm 10

Bitirme Projesinden Gelen Sonuçlar

Depodaki iki Jupyter defteri bitirme projesine aittir ve **farklı bir model, farklı bir normalizasyon uzayı ve farklı bir LIME uygulaması** kullanır. Buradaki sayılar bu raporun geri kalanıyla *karşılaştırılamaz*; arka plan olarak verilmiştir.

10.1 `Transformer.ipynb` — TranSenseFusers

Mimari. Temporal CNN + 8 başlı çapraz dikkat (attention dim 64), 14 kanal $\times$ 7000 örnek (10 s @ 700 Hz), stride 3500 (%50 örtüşme), focal loss (α=0,75, γ=2), Adam lr=1e-3, patience 25, seed 42.

Protokol. `TEST_SUBJECT = 'S3'`, `VAL_SUBJECT = 'S16'`, kalan 13 denek eğitim. 27. epoch'ta erken durdu.

	kesinlik	duyarlılık	F1	n
Stres değil	0,9046	0,9913	0,9459	459
Stres	0,9518	**0,6220**	0,7524	127
doğruluk			**0,9113**	586
makro ort.	0,9282	0,8067	0,8492	
ağırlıklı	0,9148	0,9113	**0,9040**	

Tablo 10.1: TranSenseFusers, S3 test sonucu. Doğrulama F1'i 1,0000 (S16).

Uyarı

%91,13'lük doğruluk büyük ölçüde *stres-olmayan* sınıftan geliyor: stres duyarlılığı yalnızca 0,6220, yani gerçek streslerin %38'i kaçırılıyor. Bu, "en iyi model" iddiasını tek başına doğruluk üzerinden kurmanın neden yanıltıcı olduğunun bir örneğidir.

10.2 `XAİ_metric_transensefusıon.ipynb` — offline XAI kalitesi

Kurulum. TranSenseFusers modeli, 100 örnek (50 stres + 50 stres-olmayan), üç yöntem: SHAP, LIME ve Integrated Gradients. LIME 700 örneğe indirgenip sonra geri yükseltilmiş.

Yöntem	Ins-AUC ↑	Del-AUC ↓	Monotonic ↓	Sensitivity ↓	Gini
SHAP	0,4048	0,2613	**0,4047**	**0,0038**	0,6150
LIME	0,4282	0,2153	0,5079	0,0116	**0,9949**
IG	**0,4402**	**0,2055**	0,5300	0,0053	0,6127

Tablo 10.2: Bitirme projesinin XAI kalite tablosu. Süre ölçülmemiştir.

Bu tablo bu raporun tablolarıyla karşılaştırılamaz — dört sebep

1. **Farklı model** (1,86M parametreli transformer vs 49,9K CNN).
2. **Farklı normalizasyon uzayı** (denek bazlı/sızıntılı vs nedensel).
3. **Farklı LIME uygulaması** (standart `lime` paketi + downsampling vs bölge tabanlı özel uygulama).
4. **Monotonicity tanımı ters!** Orada "düşük iyi" (ihlal sayısı), burada "yüksek iyi" (azalmayan adım oranı). Aynı isim, farklı şey.

Ayrıca sensitivity ölçeği yaklaşık 10 kat farklı (0,004–0,012 vs 0,045–0,128). İki tabloyu yan yana koymak, hakemi "hangisi doğru" sorusuna davet eder. Makalede bitirme XAI tablosunun **yer almaması** öneriliyor; tek cümlelik atıf yeterli.

Ölçülen süre bilgisi (defter çıktısından). 100 örnek için: SHAP ≈0,6 s (177 örnek/s), LIME 2 dk 37 s (1,57 s/örnek), IG 33 s (3,07 örnek/s) — GPU'da. Bu, bu raporun ana bulgusuyla *niteliksel olarak* uyumludur: açıklama, çıkarımdan kat kat pahalıdır.

Bölüm 11

Geçersiz Sayılan Ölçümler ve Nedenleri

`results/eski_ve_hatali/` klasöründeki dosyalar silinmedi: bir ölçümün neden geçersiz olduğunu gösteren kayıt da bir kanıttır. "Neden bunu kullanmıyoruz" sorusu makale yazarken ve hakem cevabında tekrar tekrar karşına çıkacak.

11.1 Saat kayması düzeltilmeden önceki sis koşusu

`fog_fog_1user.json`: **latency medyanı −8,75 ms**. Paket gönderilmeden önce varamayacağı için bu değer fiziksel olarak imkânsızdır.

Neden. Saat farkı yalnızca bağlantı başında bir kez ölçülüyordu. İki makinenin kristali aynı hızda tiklemediği için fark zamanla kaydı; 87 dakikalık bu koşuda kayma onlarca milisaniyeye ulaştı.

Düzeltme. Saat farkı artık 60 saniyede bir yeniden ölçülüyor ve her paketin latency'si iki ölçüm arasında doğrusal ara değerle düzeltiliyor (`--clock-resync-s`).

Kısmen geçerli. Aynı saatten ölçülen metrikler etkilenmedi: execution 3,62 ms, response 8,35 ms, 187 kbps, doğruluk %95,2, stres F1 0,881.

11.2 Sızıntılı normalizasyonla ölçülen XAI taramaları

`xai_sweep_lime.csv`, `xai_sweep_shap.csv` ve bunlardan türetilen karşılaştırma. Üç sorun:

1. **Yanlış girdi uzayı.** Pencereler deneğin *tüm* kaydının istatistiğiyle normalize ediliyordu — geleceğe sızmak. İki uzay ölçüldü: bağıl L2 farkı 0,787 (S11) ve 0,773 (S2), standart sapma 1,00'e karşı 1,25. XAI, dağıtımda görmeyeceği bir girdi dağılımı üzerinde ölçülmüş oldu.
2. **SHAP kendi gürültüsünü metriğe karıştırdı.** SHAP'ın `GradientExplainer`'ı her çağrıda rastgele örnekleme yapıyor, seed ise sabit değildi. Seed sabitlendikten sonra sensitivity 0,139'dan **0,075**'e düştü — yani ölçülen girdi duyarlılığının neredeyse yarısı, yöntemin kendi gürültüsüymüş.
3. **Isınma yok.** Ölçüm döngüsü ilk pencereden başlıyordu.

Yine de bir işe yarıyorlar: iki uzay yan yana konunca sızıntılı ölçümün XAI'yi olduğundan iyi gösterdiği görülüyor (LIME faithfulness 0,2476 → 0,2333, sensitivity 0,0174 → 0,0264). Makalede *metodolojik bir not* olarak kullanılabilir — sonuç olarak değil, karşı örnek olarak.

11.3 `prepared` yol dönemi streaming sonuçları

Altı dosya. İki sorun: (1) sinyal replay başlamadan 64 Hz'e indirilmişti, yani yeniden örnekleme maliyeti ölçüm dışındaydı (toplamın %29'u); (2) çoğunluk etiketi geçersiz olan pencereler tamamen atlanıyordu, bu da normalizasyon istatistiğinin o pencerelerde hiç güncellenmemesi demekti. Gerçek bir sunucu etiketi görmediği için böyle bir atlama yapamaz. Düzeltildikten sonra doğruluk 0,7674'ten **0,8222**'ye çıktı.

Uyarı

`README.md`'nin "Ölçülen sonuçlar (S11 + S14, tick=500ms, 1175 pencere)" tablosundaki sayılar (preprocessing 0,189 ms, inference 1,15 ms, toplam 1,27 ms) **bu eski koşulardan geliyor** ve güncel değildir. Makaleye `native` yolun sayıları girmeli.

11.4 Duman testleri

`fog_statstest.json`, `deployments_selftest.json`, `cloud_duman_testi.json`: kodun çalıştığını doğrulama koşuları, ölçüm değil. `realtime` kapalı olduğu için queuing, jitter ve bandwidth gerçek yükü yansıtmaz; koşu uzunlukları da rapor için yetersiz.

Bölüm 12

Tespit Edilen Tutarsızlıklar

Bu rapor hazırlanırken, `results/README.md` ile ham JSON/CSV dosyaları karşılaştırıldı. Aşağıdaki farklar bulundu. **Hepsinde JSON/CSV dosyaları esas alınmıştır**, çünkü onlar ölçümün kendisidir.

Yer	`results/README.md` ne diyor	Dosya ne diyor
`02_benchmark/ benchmark_xai_cpu`	"LIME 250 örnek p50 490 ms"	Dosyada 250 örneklik bir kayıt **yok**. İki kayıt var: 500/20/0,5 (p50 710 ms) ve 500/**5**/0,25 (p50 678 ms).
`05_yerlesim`	uç latency 0,81, execution 6,19, response 7,44 ms; sis latency 3,69, execution 3,61, response 9,97 ms	uç 0,928 / 6,344 / 7,880; sis 4,910 / 3,372 / 10,151 (üç tekrarın medyanlarının medyanı).
`01_streaming`	"1175 pencere"	`n_windows` = 2123 ölçülen pencere; etiketli pencere 1198.
`04_fog` tablosu	"execution 3,5 ms" (none)	`execution.mean_ms` = 3,586; `median_ms` = 3,530. İkisi de doğru, ama hangisi olduğu yazılmalı.
`09_secici_xai`	"hiç açıklama ort. response 15,9 ms"	Doğru, ama bu `04_fog/fog_xai_none`'un *ortalaması*; aynı README başka yerde aynı koşu için medyanı (10,6) veriyor. Karşılaştırma tablolarında ikisi karışmamalı.

Tablo 12.1: Bulunan tutarsızlıklar. Hiçbiri sonuçları değiştirmiyor, ama makaleye sayı taşınırken **doğrudan JSON'dan okunmalı**.

Öneri

`tools/paper_tables.py` yazılmalı: Bölüm 6.2 ve 7.2'deki tabloları JSON'dan LaTeX `tabular` olarak üretsin ve makaleye `\input{}` ile girsin. Elle sayı kopyalandığı sürece bu tür kaymalar tekrar olur.

Bölüm 13

Sentez: Bütün Sonuçlar Bir Arada

13.1 Sistemin tam maliyet resmi

Bileşen	Süre	Payı
Veri toplama (hesap değil)		
pencere uzunluğu	10 000 ms	—
güncelleme aralığı (hop)	5 000 ms	**bütçe**
Sunucu hesabı, açıklama kapalı (sis düğümü)		
resampling	1,05 ms	%31
preprocessing	0,17 ms	%5
tensor_prep	0,11 ms	%3
inference	2,01 ms	%59
postprocessing	0,05 ms	%1
toplam	**3,39 ms**	**bütçenin %0,07'si**
Açıklama (her pencerede)		
`shap_cheap`	395 ms	bütçenin %7,9'u
`lime_cheap`	590 ms	bütçenin %11,8'i
`lime`	1189 ms	bütçenin %23,8'i
`shap`	1422 ms	bütçenin %28,4'ü
Ağ (tek yön)		
uç (loopback)	≈1 ms	
sis (Wi-Fi)	4–5 ms	
bulut (internet)	40–60 ms	

Tablo 13.1: Uçtan uca maliyet resmi. **En büyük tek kalem, bütün ağ ve hesap bileşenlerinin toplamından yüzlerce kat büyük olan açıklamadır.**

13.2 Neden tek kullanıcıda bile önemli

Tek kullanıcıda 1422 ms'lik en pahalı açıklama bile 5000 ms bütçesine sığıyor (%28), dolayısıyla "sorun yok" denebilirmiş gibi görünür. Üç sebeple sorun vardır:

1. **Çok kullanıcıda sığmıyor.** 4 istemcide kuyruk p95 1125 ms; kapasite hep-açıkla durumunda 12 istemciye düşüyor.

2. **Zayıf donanımda sığmıyor.** Referans çalışmanın Raspberry Pi 4 worker'ında execution 34,7 ms, bu çalışmada 3,6 ms — yaklaşık **10 kat** fark. Aynı oran açıklamaya uygulanırsa `shap_cheap` bir Pi 4'te ≈**4 saniye** eder ve 5 saniyelik bütçeyi neredeyse doldurur.

3. **Kullanıcı deneyimi.** 1456 ms'lik bir yanıt, 10,6 ms'lik bir yanıtın yanında "sistem takıldı" hissi verir.

13.3 Referans makale ile karşılaştırma

Erzurumluoğlu ve ark., ICACT 2025; DREAMER veri setiyle sis mimarisinde duygu tanıma. Worker düğümü Raspberry Pi 4.

Metrik	Makale (worker)	Bu çalışma	Karşılaştırılabilir mi?
latency	4,7 ms	0,81 ms(loopback)	hayır — farklı ağ
queuing delay	<1 ms	0,045 ms	hayır
execution time	34,7 ms	7,18 ms	hayır — Pi 4 vs masaüstü
response time	43,3 ms	8,82 ms	hayır
jitter	17,4 ms	3,53 ms	hayır
bandwidth	**374 kbps**	**187 kbps**	**evet**

Tablo 13.2: Yalnızca bant genişliği doğrudan karşılaştırılabilir.

Anlamlı olan tek karşılaştırma

Onlar 3 saniyelik pencereden 24 öznitelik çıkarıp gönderiyor, biz 10 saniyelik pencerenin **ham sinyalini** gönderiyoruz — buna rağmen **yarısı kadar** veri geçiyor.
Asıl fark mimari: onların sisteminde öznitelik çıkarımı hesabın çoğunu oluşturuyor, bizde ham sinyal doğrudan modele giriyor ve hesap çok daha ucuz. Buna karşılık bizim uçtan uca gecikmemizi *hesap değil pencere uzunluğu* belirliyor: 10 saniyelik pencere, 8,8 ms yanıt süresinin bin katından fazla.
Broker neden gerekmiyor. Onların mimarisinde broker, istekleri en az yüklü worker'a dağıtan bir yönlendiricidir. Tek worker varken dağıtacak bir şey yoktur. Kendi sonuçlarında da broker ile worker sayıları neredeyse aynı çıkmış (43,9'a karşı 43,3 ms).

Bölüm 14

Sınırlılıklar

1. **İki tutulmuş denek (n=2).** Kalibrasyon bulgusu ve "denek bağımlı" sonucu yalnızca S11 ve S14 üzerine kurulu. Bu bir genelleme değil, bir gözlemdir.

2. **Sis düğümü bir masaüstü, Raspberry Pi değil.** Ölçülen şey *dağıtık yerleşim*dir, *kaynak kısıtlı sis düğümü* değil. Ölçüldü ki tek çekirdeğe kilitlemek bu iş yükünde hiçbir şey değiştirmiyor: süreç zaten tek iplik ve bir çekirdeği aşmıyor (tepe CPU %105–111). Bağlayıcı olan tek yöntem kesirli CPU kotasıdır (Docker `--cpus=0.1`).

3. **Yazılım yığını yerleşimler arasında farklı.** Uç: Python 3.11 / PyTorch 2.8. Sis: Python 3.14 / PyTorch 2.14. Bulut: Docker python:3.11 / PyTorch 2.14. Bu, 07_edge_xai ile 04_fog arasındaki tutarsız sıralamanın sebebidir. Kapatmak için sis düğümünü Docker'la yeniden koşmak yeterli ($\approx$25 dk).

4. **Kuyruk ölçümü bir üst sınırdır.** İstemciler eşzamanlı başlatıldığı için faz hizalı kalıyor. Gerçek dağıtımda rastgele fazlarda başlarlar.

5. **Sis XAI koşuları 44 pencere, tek tekrar.** Ortalama karşılaştırması için yeterli; p95/p99 yalnızca 1030 pencerelik seçici koşudan alınmalı.

6. **Kullanıcı çalışması yok.** Açıklamaların insanlara gerçekten yardım edip etmediği ölçülmedi. Bu standart bir XAI sınırlılığıdır.

7. **Tek model, iki açıklayıcı.** Model karmaşıklığı ablasyonu (aynı mimarinin küçük/büyük varyantları) ve Integrated Gradients kapsam dışı.

8. **Latency Wi-Fi'da ve internette belirsiz.** Tek yönlü gecikmenin hata payı Wi-Fi'da $\pm$1–1,7 ms, internette $\pm$25–51 ms. Execution, queuing (sunucu saati), response ve jitter (istemci saati) bundan etkilenmez.

9. **Emülasyon, canlı sistem değil.** WESAD kaydından replay yapılıyor. Bu, referans makalenin de yaptığı şeydir ve yöntemin savunulabilirliği buna dayanır — ama "clinical" ya da "live system" denmemelidir.

Bölüm 15

Ekler

15.1 Her sayı hangi dosyadan

Sonuç	Kaynak dosya
Aşama bazlı hesaplama profili (tek süreç)	`01_streaming/streaming_summary_native_fixed.json`
İplik taraması, koşu arası yayılım	`02_benchmark/benchmark_thread_sweep.json`
`prepared` vs `native`	`02_benchmark/benchmark_pipeline_cmp.json`
Açıklayıcıların istemci CPU maliyeti	`02_benchmark/benchmark_xai_cpu.json`
Yedi metrik, loopback	`03_loopback/fog_realtime_1user.json`
Kuyruk metriğinin doğrulanması	`03_loopback/fog_full_queue.json`
Ana maliyet tablosu (sis)	`04_fog/fog_xai_{none,shapucuz, ewline limeucuz,lime,shap}.json`
Uç vs sis (XAI'siz)	`05_yerlesim/deployments_edge_vs_fog.json`
XAI hiperparametre taraması	`06_xai_tarama/xai_sweep_{lime,shap}_causal.csv`
Ortak yerleşimde XAI	`07_edge_xai/edge_xai_*.json`
Kalibrasyon, ECE, eşik tablosu	`08_kalibrasyon/calibration_d1.json`
Seçici açıklama, ana koşu	`09_secici_xai/fog_sel80_tam.json`
Eşik projeksiyonu (48 satır)	`09_secici_xai/selective_xai_sweep_d2.json`
Eşiğin tetiklenmediği koşular	`09_secici_xai/kisa_segment_sel{60,80,90}.json`
Çok istemcili yük ve kapasite	`10_yuk/fog_load_{none,shapucuz}_{2,4}.json`
Kalite–gecikme Pareto	`11_pareto/pareto_c5.json`
Dört kaynaktan aşama derlemesi	`12_profil/profile_table_b1.json`
Bulut yerleşimi	`13_bulut/cloud_{taban_3tur,xai_*,load_4}.json`
Altı şekil	`14_figures/fig{1..6}_*.png`
Seçilmiş XAI yapılandırmaları	`checkpoints/xai_config.json`
SHAP arka plan kümesi	`checkpoints/shap_background_k100_kmeans.npz`
MCU fizibilitesi	`python -m tools.mcu_feasibility` (kayıtsız)
TranSenseFusers başarımı	`Transformer.ipynb`, hücre 10
Offline XAI kalite tablosu	`XAİ_metric_transensefusıon.ipynb`, hücre 1

15.2 Deneyleri yeniden koşma komutları

```
# 01 --- tek süreçli replay
python run_streaming_emulation.py --subject S11 S14 --tag native_fixed

# 02 --- tekrarlı, dönüşümlü ölçüm
python benchmark_runs.py --subject S11 --repeat 4 --threads 1 2 4 --tag thread_sweep
python benchmark_runs.py --subject S11 --repeat 4 --pipeline prepared native --tag pipeline_cmp

# 03 --- loopback, yedi metrik
python run_fog_experiment.py --subject S11 --realtime --tag realtime_1user

# Sunucuyu ikinci makinede başlat (XAI ile)
python -m serving.server --host 0.0.0.0 --port 9099 \
    --xai-method shap_cheap --xai-threshold 1.0

# 04 --- sis düğümünde XAI (her durum için ayrı)
python run_fog_experiment.py --server-host 192.168.1.14 --subject S11 \
    --realtime --max-ticks 600 --tag xai_shapucuz

# 05 --- yerleşim karşılaştırması
python compare_deployments.py --subject S11 --realtime --max-ticks 600 --repeat 3 \
    --deployment edge=local fog=192.168.1.14 --tag edge_vs_fog

# 08 --- kalibrasyon
python -m tools.calibration --tag d1

# 09 --- seçici XAI (sunucuda --xai-threshold 0.8 ile)
python run_fog_experiment.py --server-host 192.168.1.14 --subject S11 \
    --realtime --tag sel80_tam
python -m tools.selective_sweep --tag d2

# 10 --- çok istemcili yük
python run_fog_experiment.py --server-host 192.168.1.14 --subject S11 S14 \
    --clients 4 --realtime --max-ticks 600 --tag load_4

# 11, 12, 14 --- türetilmiş çıktılar
python -m tools.pareto --tag c5
python -m tools.profile_table
python -m tools.figures

# Donanımsız MCU analizi
python -m tools.mcu_feasibility
```

15.3 Docker ile sunucuyu dağıtma

```
# Sunucu makinesinde
WESAD_XAI_METHOD=shap_cheap WESAD_XAI_THRESHOLD=0.8 \
docker compose up -d --build
docker compose logs -f stress-server

# Sunucu WESAD verisine İHTİYAÇ DUYMAZ --- veri istemcide kalır.
```

15.4 Bu raporu yeniden derleme

```
cd docs
xelatex teknik_rapor.tex
xelatex teknik_rapor.tex    # içindekiler için ikinci geçiş
```

Şekiller, depo kökündeki `results/14_figures/` ve `results/11_pareto/` klasörlerinden okunur. Bu şekilleri üreten betikler yeniden koşulursa rapor da güncel hâllerini alır.